

Thesis for the Degree of Master of Computer Science (MCS)

**Comparative Benchmarking of CatBoost, TabPFN,
and TabFM for Explainable Multi-Tier Hair Fall
Risk Stratification**



Krishna Gautam

(LC ID: LC0015002874)

Lincoln International College of Management & IT

Lincoln University College, Malaysia

September, 2026

Research Project for the Degree of Master of Computer Science
(MCS)

**Comparative Benchmarking of CatBoost, TabPFN,
and TabFM for Explainable Multi-Tier Hair Fall
Risk Stratification**

Supervised by Rabin Thapa

A thesis submitted in partial fulfilment of the requirements for the
degree of Master of Computer Science (MCS)

Krishna Gautam

(LC ID: LC0015002874)

Lincoln International College of Management & IT

Lincoln University College, Malaysia

September, 2026

Declaration

I hereby declare that this study entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification** is based on my original research work. Related works on the topic by other researchers have been duly acknowledged. I owe all the liabilities relating to the accuracy and authenticity of the data and any other information included hereunder.

.....

Signature

Name of the Student: Krishna Gautam

Registration Number: LC ID: LC0015002874

Date:

Recommendation

This is to certify that this thesis entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification**, prepared and submitted by **Krishna Gautam**, in partial fulfilment of the requirements of the degree of Master of Computer Science (MCS) awarded by Lincoln University College, Malaysia, has been completed under my supervision. I recommend the same for acceptance by Lincoln University College, Malaysia.

.....

Signature

Name of the Supervisor: Rabin Thapa

Organization: Lincoln International College of Management & IT

Date:

Certificate

This thesis entitled **Comparative Benchmarking of CatBoost, TabPFN, and TabFM for Explainable Multi-Tier Hair Fall Risk Stratification** prepared and submitted by **Krishna Gautam** has been examined by us and is accepted for the award of the degree of Master of Computer Science (MCS) by Lincoln University College, Malaysia.

.....

Signature:

External Examiner

Date signed:

Rabin Thapa

Signature:

Supervisor

Date signed:

.....

Signature:

Head of Department / Principal

Date signed:

Acknowledgements

I would like to express my sincere gratitude to my supervisor, Rabin Thapa, for his guidance, suggestions and encouragement throughout this research. His comments at every stage helped me to shape the study and to improve this thesis.

I am thankful to Lincoln International College of Management & IT and to all the faculty members and staff of the Master of Computer Science programme for their teaching and support during my studies. I also thank the company for providing its dataset for this study, and the developers of the open-source software used in the experiments.

I am grateful to my classmates and friends for their discussions and help, and to my family for their patience, support and encouragement.

.....

Signature

Name of the Student: Krishna Gautam

Registration Number: LC ID: LC0015002874

Date:

List of Abbreviations/Acronyms

ALT	Alanine Aminotransferase
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
CV	Cross-Validation
EHR	Electronic Health Record
GPU	Graphics Processing Unit
KNN	k-Nearest Neighbour
MAE	Mean Absolute Error
MCS	Master of Computer Science
NeurIPS	Conference on Neural Information Processing Systems
OOM	Out of Memory
QWK	Quadratic Weighted Kappa
ROC-AUC	Receiver Operating Characteristic – Area Under the Curve
SHAP	Shapley Additive Explanations
SVM	Support Vector Machine
TabFM	Tabular Foundation Model
TabPFN	Tabular Prior-Fitted Network
VRAM	Video Random Access Memory

Units and Conversions

%	Percent
g/dL	Grams per decilitre
GB	Gigabyte
h	Hour
mg/dL	Milligrams per decilitre
min	Minute
ng/mL	Nanograms per millilitre
s	Second
U/L	Units per litre
µg/dL	Micrograms per decilitre
µg/L	Micrograms per litre

ABSTRACT

Hair loss is influenced by many factors acting together, and no single measurement predicts it reliably. Earlier prediction studies used classical classifiers, reached conflicting conclusions about the best model, and did not test tabular foundation models. This study benchmarked a tuned gradient-boosted model, CatBoost, against two zero-shot tabular foundation models, TabPFN and TabFM, for classifying hair fall risk into three tiers (Low, Moderate and High) from structured clinical and lifestyle data, and explained all three models with SHAP.

The dataset had 21,606 records with 20 predictors, taken from a company dataset. A stratified 80/20 split gave 17,284 training and 4,322 test records. Each model was run with 500, 2,000 and all 17,284 training rows and scored on the same test records. CatBoost was tuned by grid search with five-fold cross-validation, while TabPFN and TabFM received the training rows as context without tuning. The models were compared using accuracy, macro-averaged precision, recall and F1-score, ROC-AUC and McNemar's test.

With all training rows, accuracy was 78.20% for CatBoost, 78.62% for TabPFN and 78.67% for TabFM, and no pair of models differed significantly. With 500 rows the accuracies were 69.37%, 76.89% and 76.84%, and with 2,000 rows 74.97%, 78.27% and 78.44%; at these sizes both foundation models were significantly better than CatBoost ($p < 0.0001$). Almost all errors were between neighbouring tiers, and the Moderate tier was classified least well. SHAP showed that iron and stress level were the two most influential features in all nine analyses, with the same direction of effect in every model, reflecting relationships built into the dataset, not new clinical evidence. On a GPU, predicting the test records took 290 seconds for TabPFN and 3,082 seconds for TabFM with the full context, whereas CatBoost was fitted in 3.3 seconds.

Zero-shot foundation models matched a tuned gradient-boosted model when all data were used, clearly outperformed it when data were limited, and could be explained with SHAP, but at a much higher computing cost. Because the results come from one company dataset and one data split, testing on real clinical data is recommended.

Keywords: hair fall risk, tabular foundation models, CatBoost, TabPFN, TabFM, SHAP, explainable machine learning

Table of Contents

Declaration.....	ii
Recommendation.....	iii
Certificate.....	iv
Acknowledgements.....	v
List of Abbreviations/Acronyms.....	vi
Units and Conversions.....	vii
ABSTRACT.....	viii
Table of Contents.....	ix
List of Tables.....	xii
List of Figures.....	xiv
List of Appendices.....	xv
CHAPTER 1 INTRODUCTION.....	1
1.1 Background.....	1
1.2 Statement of the problem.....	3
1.3 Research questions.....	4
1.4 Research objectives.....	5
1.5 Significance/Rationale of the study.....	6
CHAPTER 2 LITERATURE REVIEW.....	8
2.1 Machine learning for hair loss prediction.....	8
2.2 Tree ensembles and gradient boosting.....	10
2.3 Tree-based models and deep learning on tabular data.....	11
2.4 Transformers, attention and in-context learning.....	12
2.5 Tabular foundation models.....	12
2.6 Explainability with SHAP.....	13
2.7 Evaluating and comparing classifiers.....	15
2.8 Critical summary and research gap.....	16
CHAPTER 3 METHODOLOGY.....	18
3.1 Research design.....	18
3.2 Research approach.....	19
3.3 Study area.....	20
3.4 Study population.....	20
3.5 Sample selection.....	20
3.6 Sample size.....	20
3.7 Methods of data collection.....	21
3.8 Data analysis approach and tools.....	22

3.8.1 Data preprocessing.....	22
3.8.2 Descriptive analysis of the dataset.....	23
3.8.3 Models and justification.....	23
3.8.4 Experimental setup.....	25
3.8.5 Training size experiment.....	26
3.8.6 Explainability analysis.....	26
3.8.7 Evaluation metrics and statistical testing.....	27
3.8.8 Computational assessment.....	28
3.8.9 Tools and environment.....	29
3.9 Ethical considerations.....	29
3.10 Validity, reliability and reproducibility.....	30
CHAPTER 4 RESULTS AND DISCUSSION.....	31
4.1 Predictive performance of the three models.....	31
4.1.1 Overall performance.....	31
4.1.2 Per-class performance and confusion matrices.....	32
4.1.3 Statistical significance.....	34
4.1.4 Quality of the predicted probabilities.....	36
4.1.5 Overlap of the errors of the three models.....	37
4.1.6 Performance in subgroups.....	37
4.2 Effect of training context size.....	38
4.3 Explainability of the predictions.....	39
4.3.1 Relationship between the features and the risk tier in the data.....	39
4.3.2 Global feature importance.....	40
4.3.3 Direction of the effects.....	43
4.3.4 Differences between the rankings.....	45
4.4 Computational cost and implementation limitations.....	46
4.4.1 Run time.....	46
4.4.2 Software problems and workarounds.....	48
4.5 Discussion.....	49
4.5.1 Plausibility of the findings.....	49
4.5.2 Comparison with similar studies.....	49
4.5.3 Implications and cautions.....	50
4.5.4 Relation of the results to the research questions.....	50
4.5.5 Threats to the validity of the findings.....	51
CHAPTER 5 CONCLUSION AND RECOMMENDATIONS.....	53
5.1 Conclusion.....	53
5.2 Limitations of the study.....	55
5.3 Recommendations.....	55

REFERENCES.....	58
Appendix A: Source Code.....	61
Appendix B: Additional Figures.....	78
Appendix C: Result Tables from the Experiments.....	85
Appendix D: Dataset Description and Sample Records.....	88

List of Tables

Table 2.1	Summary of related studies on hair loss prediction.....	9
Table 2.2	Model families relevant to this study.....	16
Table 3.1	Link between the research objectives and the methods used.....	18
Table 3.2	Predictor variables.....	21
Table 3.3	Descriptive statistics of the continuous predictors (all 21,606 records).....	22
Table 3.4	Biomarkers compared with clinical reference ranges (all 21,606 records)...	23
Table 3.5	Settings of the three models.....	25
Table 3.6	Experimental environment.....	29
Table 4.1	Performance of the three models on the test partition (4,322 records) at each training size.....	31
Table 4.2	Hyperparameters chosen for CatBoost by cross-validation.....	32
Table 4.3	Precision, recall and F1-score by risk tier (full training data).....	32
Table 4.4	Errors of each run and their distance from the true tier.....	34
Table 4.5	Ordinal-aware measures for the nine runs.....	34
Table 4.6	Pairwise McNemar's test at each training size (4,322 test records).....	35
Table 4.7	Accuracy (%) with 95% Wilson score confidence intervals.....	35
Table 4.8	Log loss, Brier score and mean confidence of the nine runs.....	36
Table 4.9	Number of test records classified correctly by all, two, one or none of the three models.....	37
Table 4.10	Accuracy (%) in subgroups of the test partition.....	37
Table 4.11	Accuracy (%) and macro-F1 by training size.....	38
Table 4.12	F1-score by risk tier and training size.....	39
Table 4.13	Mean of the continuous predictors by risk tier and Spearman correlation with the tier.....	40
Table 4.14	Share of records (%) with each binary flag, by risk tier.....	40
Table 4.15	Ten most important features with the full training data, by mean absolute SHAP value.....	42
Table 4.16	Five most important features in each of the nine SHAP analyses.....	43
Table 4.17	Spearman rank correlation of the SHAP feature rankings of the three models (full training data).....	45
Table 4.18	Fit and prediction time of the nine runs (seconds).....	46
Table 4.19	Time of the SHAP analyses.....	48
Table C.1	Results of all model runs (final_results.csv from Step 10).....	85

Table C.2	CatBoost grid search with 500 training rows (5-fold cross-validation, sorted by macro-F1).....	85
Table C.3	CatBoost grid search with 2,000 training rows (5-fold cross-validation, sorted by macro-F1).....	85
Table C.4	CatBoost grid search with 17,284 training rows (5-fold cross-validation, sorted by macro-F1).....	86
Table C.5	Precision, recall and F1-score by risk tier with 500 and 2,000 training rows	86
Table C.6	Mean absolute SHAP value of every feature with the full training data (average over the three classes).....	86
Table C.7	Settings and run time of the nine SHAP analyses.....	87
Table D.1	First three records of the dataset (full_name omitted; one column per record)	88

List of Figures

Fig. 3.1	Block diagram of the research methodology.....	19
Fig. 4.1	Confusion matrices of the three models with the full training data.....	33
Fig. 4.2	Accuracy, macro-F1 and ROC-AUC of the three models against the number of training rows (logarithmic axis).....	38
Fig. 4.3	CatBoost global feature importance (SHAP), full training data.....	41
Fig. 4.4	TabPFN global feature importance (SHAP), full training data.....	41
Fig. 4.5	TabFM global feature importance (SHAP), full training data.....	42
Fig. 4.6	CatBoost SHAP beeswarm plot for the High risk class, full training data....	44
Fig. 4.7	TabPFN SHAP beeswarm plot for the High risk class, full training data....	44
Fig. 4.8	TabFM SHAP beeswarm plot for the High risk class, full training data.....	45
Fig. 4.9	Fit and prediction time of the three models at each training size (logarithmic axis). The hatched bar has no prediction time (resumed run).....	47
Fig. B.1	Confusion matrices with 500 training rows.....	78
Fig. B.2	Confusion matrices with 2,000 training rows.....	79
Fig. B.3	Global feature importance (SHAP) with 500 training rows.....	80
Fig. B.4	Global feature importance (SHAP) with 2,000 training rows.....	81
Fig. B.5	SHAP beeswarm plots for the High risk class with 500 training rows.....	82
Fig. B.6	SHAP beeswarm plots for the High risk class with 2,000 training rows.....	83
Fig. B.7	Class distribution of the hair fall risk tiers in the dataset.....	83
Fig. B.8	Share of records outside the clinical reference range, by biomarker.....	84

List of Appendices

Appendix A: Source Code.....	61
Appendix B: Additional Figures.....	78
Appendix C: Result Tables from the Experiments.....	85
Appendix D: Dataset Description and Sample Records.....	88

CHAPTER 1

INTRODUCTION

This chapter introduces the study. It presents the background of hair fall risk prediction (Section 1.1), the statement of the problem (Section 1.2), the research questions (Section 1.3), the research objectives (Section 1.4) and the significance of the study (Section 1.5). The thesis has five chapters. Chapter 2 reviews the literature, Chapter 3 describes the methodology, Chapter 4 presents and discusses the results, and Chapter 5 gives the conclusion and recommendations. The references and appendices follow Chapter 5.

Scope of the study. The study is limited to three-tier (Low, Moderate, High) hair fall risk classification from structured tabular clinical and lifestyle data. The dataset has 21,606 records, taken from a company dataset. Three models are compared: CatBoost, TabPFN and TabFM. Each is trained on, or supplied as context with, 500, 2,000 and all 17,284 training records, and each is evaluated on the same held-out test partition of 4,322 records with accuracy, macro-averaged precision, recall and F1-score, ROC-AUC (area under the receiver operating characteristic curve) and McNemar's test. The predictions of every model at every training size are explained with SHAP (Shapley Additive Explanations).

Limitations of the study. The dataset came from a single company and was not drawn from a clinical registry, so the results show model capability on this data and are not a validated diagnostic tool. All results come from one train/test split and one random seed. The SHAP analysis of the two foundation models used a small sample of test records (30 for TabPFN and 20 for TabFM) because each explanation needs thousands of model calls. Scalp images, free-text records and other hair or scalp conditions are outside the scope of the study.

1.1 Background

Hair loss, known clinically as alopecia, affects people of all ages and backgrounds. Some daily shedding is a normal part of the hair growth cycle. A marked or sudden increase in shedding, however, often signals a deeper disturbance such as hormonal imbalance, nutrient deficiency, physical strain or prolonged psychological stress. Visible hair loss also carries an emotional cost, and it is commonly linked to lower self-confidence, anxiety and a reduced sense of well-being.

Hair grows in cycles. Each follicle passes through a growth phase, a short transition phase and a resting phase, after which the hair is shed and a new hair begins to grow. In a healthy scalp only a small share of hairs is shed each day. Hair fall becomes a concern when more follicles than usual enter the resting phase and are shed together. This can be triggered by physical or psychological stress, inadequate nutrition, illness and hormonal change, so very different conditions can produce the same visible result. This is one reason why no single test identifies the risk, and why an estimate based on several measurements together is more informative than a judgement based on one.

Predicting who is at risk is difficult because no single biomarker reliably indicates hair loss. Risk arises from the combined and non-linear interaction of many factors, including iron and protein levels, vitamin D status, liver function, stress, sleep habits, exposure to chemical hair treatments, chronic illness and family history. Because these factors act together, it is impractical for a clinician or an individual to judge overall risk by observation alone. In practice, assessment still depends largely on self-reported history and delayed clinical examination, which are subjective, inconsistent between observers and hard to reach in under-resourced settings.

Much of the data that could support a more objective assessment is tabular. Laboratory results, demographic details, clinical history and lifestyle indicators are naturally stored as rows and columns, and Electronic Health Records (EHRs) hold such data in large volumes. Machine learning can find patterns in this kind of data that are hard to notice manually, and several studies have applied it to hair loss. Widowati et al. [1] built an explainable decision-support system for multi-class hair loss severity classification and showed that feature transformations must be restricted to within cross-validation folds to avoid inflated performance estimates. Sayyad and Midhunchakkaravarthy [2] developed a diagnostic model for Alopecia Areata, but it addressed a single condition rather than the wider set of risk factors behind hair loss. Sai et al. [3] compared individual classifiers with an ensemble method for hair fall prediction and found that the ensemble performed better on accuracy, precision and recall. Patel et al. [4] proposed a Random Forest framework using hereditary, hormonal, nutritional and medical-history factors. Chen et al. [5] combined genetic, hormonal, scalp and lifestyle data and observed that neural networks captured more complex feature interactions but were harder to interpret. Image-based work has also been done: Shakeel et al. [6] extracted texture, shape and colour features from scalp images and

classified healthy hair and Alopecia Areata with Support Vector Machine (SVM) and k-Nearest Neighbour (KNN) classifiers.

On structured data, gradient-boosted decision trees have long been the standard approach. CatBoost [7] is one such method, designed to handle categorical features and to reduce target leakage through ordered target encoding. A newer family of models, tabular foundation models, takes a different route. These are transformers pretrained on large collections of synthetic datasets, and they predict directly from the training data supplied as context in a single forward pass, without dataset-specific training. TabPFN [8] introduced this idea, and TabFM [9], released by Google Research, extends it with attention applied alternately across rows and columns. Reports on the TabArena benchmark suggest that TabFM can outperform heavily tuned gradient-boosted models in a purely zero-shot setting [9], and an independent reproduction has confirmed its competitive accuracy while also documenting defects in the official implementation [10].

Predictive accuracy alone is not enough in a health setting, because a practitioner needs to know why a model reached its output. Both boosted ensembles and transformer models are opaque by default. SHAP (Shapley Additive Explanations) [11] addresses this by attributing a prediction to individual input features using Shapley values from cooperative game theory, and it can be applied to very different model types. This makes it suitable for comparing explanations across architectures.

The literature reviewed above shows that hair loss prediction has mostly relied on classical classifiers, that interpretability is a recognised concern for the more complex models, and that tabular foundation models have not been evaluated against gradient boosting on hair fall data. To our knowledge, no study has compared more than one foundation model on this task. These gaps motivate the present study, which benchmarks CatBoost, TabPFN and TabFM for multi-tier hair fall risk classification (Low, Moderate and High) on structured clinical and lifestyle data and applies SHAP to all three models.

1.2 Statement of the problem

Hair fall risk is hard to assess objectively. It depends on the combined effect of nutritional, hormonal, metabolic, lifestyle and hereditary factors, and no single indicator is reliable on its own (Section 1.1). Current practice relies on self-reported history and delayed clinical examination, which vary between observers and are difficult to access in under-resourced

settings. Machine learning can model these interacting factors, but the existing work has three shortcomings that make it inadequate for this purpose.

First, the evidence on which model suits hair fall data is inconsistent. Sai et al. [3] found that an ensemble method outperformed individual classifiers, whereas Patel et al. [4] found that Random Forest outperformed XGBoost, CatBoost and LightGBM. Studies also differ in target (a single condition such as Alopecia Areata [2], image-based classification [6], or survey-based severity [1]), so their results cannot be compared directly. No clear, well-founded choice of model exists for structured hair fall data.

Second, tabular foundation models have not been tested on this problem. TabPFN [8] and TabFM [9] predict from supplied context without dataset-specific training, and TabFM has been reported to outperform heavily tuned gradient-boosted trees on the TabArena benchmark [9]. That evidence comes from general benchmarks, not from health survey data. It is therefore unknown whether these models can match a tuned gradient-boosted model such as CatBoost [7] on hair fall risk, or whether the result holds across more than one foundation model. The reliability of a newly released model such as TabFM also needs checking, since an independent reproduction found defects in its official implementation [10].

Third, the models that capture complex feature interactions are difficult to interpret. Chen et al. [5] observed that neural models gained accuracy at the cost of per-prediction interpretability, and the same opacity applies to boosted ensembles and transformers. Without feature-level explanations, a predicted risk level cannot be checked against known clinical reasoning, and this limits its usefulness for decision support.

These gaps make the research necessary. A comparison of CatBoost, TabPFN and TabFM on the same data, split and metrics, with SHAP explanations for all three, is needed to establish whether tuning-free foundation models can replace tuned gradient boosting for multi-tier (Low, Moderate, High) hair fall risk classification, and whether their predictions can be traced to interpretable clinical and lifestyle indicators.

1.3 Research questions

Based on the problem stated in Section 1.2, this study seeks to answer the following research questions:

1. Can the zero-shot tabular foundation models TabPFN and TabFM match the predictive performance of a tuned CatBoost model in classifying multi-tier (Low, Moderate, High) hair fall risk from structured clinical and lifestyle data, and are the differences between the three models statistically significant?
2. How does the amount of training data supplied (the training context size) affect the performance of CatBoost, TabPFN and TabFM, and which model uses limited data most efficiently?
3. Which clinical and lifestyle features contribute most to the predicted risk tier in each model, and are these contributions consistent across the three models and clinically plausible?
4. What are the practical computational costs and implementation limitations (runtime, memory and software reliability) of using each model, particularly the recently released TabFM?

1.4 Research objectives

General objective

To benchmark CatBoost, TabPFN and TabFM for explainable multi-tier (Low, Moderate, High) hair fall risk stratification from structured clinical and lifestyle data.

Specific objectives

1. To compare the predictive performance of a tuned CatBoost model with that of the zero-shot TabPFN and TabFM models using accuracy, macro-precision, macro-recall, macro-F1 and ROC-AUC, and to test whether the differences between them are statistically significant.
2. To evaluate how the training context size affects the performance of the three models, and to identify which model uses limited training data most efficiently.
3. To explain the predictions of the three models with SHAP and to compare the features that contribute most to the predicted risk tier across the models.
4. To assess the computational cost and implementation limitations of the three models, with particular attention to the recently released TabFM.

1.5 Significance/Rationale of the study

This study is justified by the gaps identified in Section 1.2. It compares a tuned gradient-boosted model with two tabular foundation models on the same hair fall data and explains all three with SHAP. The main beneficiaries and the ways they benefit are as follows.

Clinicians and dermatology practitioners. A model that estimates Low, Moderate or High hair fall risk from routine laboratory values and lifestyle information can support earlier screening and a more consistent assessment than self-reported history alone. Because SHAP attributes each prediction to specific indicators such as iron, protein, vitamin D and stress level, a practitioner can check the result against clinical knowledge instead of accepting it as a black box. The study is intended as decision support, not as a replacement for clinical diagnosis.

Individuals at risk of hair loss. Risk tiers linked to identifiable, modifiable factors (for example nutritional status, stress and sleep habits) can help people recognise early warning signs and seek advice sooner. This is particularly useful where access to specialist dermatological care is limited.

Machine learning researchers and practitioners working with tabular data. The study gives a controlled comparison of two independently developed foundation models, TabPFN and TabFM, against a tuned CatBoost baseline. The context-size analysis shows how much training data each model needs, which matters where labelled clinical data is scarce. The record of TabFM's runtime behaviour and implementation problems helps others decide whether to adopt this recently released model.

Future researchers. The study uses one dataset, split, set of metrics and explanation method for all three models, and it reports the limits of the data and of the computing resources available. This provides a reproducible reference for later work, such as evaluation on real clinical data or ordinal classification methods.

Beyond these groups, the study has value for the research community as a documented example of how a new class of models can be evaluated fairly on a health problem. It shows the steps that a fair comparison needs: a tuned baseline, the same data and split for every model, a statistical test, a variation of the training size and an explanation method applied to every model. It also records the practical problems that arose with a recently released model, which are usually left out of benchmark reports.

Overall, the study helps establish whether tuning-free foundation models can replace tuned gradient boosting for structured health risk prediction while keeping predictions explainable.

CHAPTER 2

LITERATURE REVIEW

This chapter surveys published work relevant to the study. It first reviews machine learning studies on hair loss (Section 2.1). It then reviews the three families of methods used in this study: tree ensembles and gradient boosting (Section 2.2), the debate on trees versus deep learning for tabular data (Section 2.3), and transformer-based tabular foundation models (Sections 2.4 and 2.5). Section 2.6 reviews explainability with SHAP, and Section 2.7 reviews the measures and tests used to compare classifiers. Section 2.8 brings the literature together, criticises it and identifies the gap that this study addresses.

2.1 Machine learning for hair loss prediction

Machine learning has been applied to hair loss with structured survey data, clinical data and scalp images. Studies using structured data are the most relevant here because the present study also uses tabular clinical and lifestyle features.

Widowati et al. [1] developed an explainable decision-support system for multi-class hair loss severity classification. Their main methodological point was that feature transformations should be fitted only inside cross-validation folds, since fitting them on all the data inflates the estimated generalisation performance. This is a useful warning for any comparison of models, because leakage from preprocessing can favour one model over another without being noticed. Sayyad and Midhunchakkaravarthy [2] built a diagnostic model for Alopecia Areata and improved prediction for that condition, but their model did not cover the wider multi-factorial risk drivers of hair loss.

Several studies compared classifiers. Sai et al. [3] compared individual classification algorithms with an ensemble method for hair fall prediction and reported that the ensemble outperformed every individual algorithm on accuracy, precision and recall. Patel et al. [4] proposed a Random Forest framework using hereditary factors, hormonal markers, medical history and nutritional status, and found that Random Forest outperformed XGBoost, CatBoost and LightGBM on their dataset. Anderson and Roberts [12] analysed survey-based lifestyle and clinical indicators, including dandruff severity, chronic illness history, thyroid function and micronutrient status, and used ROC-AUC and macro-F1 to account for class imbalance. Chen et al. [5] combined genetic, hormonal and scalp-condition data with lifestyle variables and compared standard classifiers with neural networks. The neural

networks captured more complex feature interactions, but at the cost of interpretability of individual predictions.

Image-based approaches have also been reported. Shakeel et al. [6] extracted texture, shape and colour features from scalp images and classified healthy hair and Alopecia Areata with Support Vector Machine (SVM) and k-Nearest Neighbour (KNN) classifiers, obtaining 91.4% accuracy for SVM against 88.9% for KNN. Image-based work answers a different question from the present one, since it detects a visible condition, whereas risk stratification from laboratory and lifestyle data aims to estimate risk before hair loss is visible.

Table 2.1 summarises these studies.

Table 2.1: Summary of related studies on hair loss prediction

Ref.	Focus	Approach	Main finding	Limitation relevant to this study
[1]	Multi-class hair loss severity	Explainable decision-support system	Transformations must be kept inside cross-validation folds	Does not compare foundation models
[2]	Alopecia Areata	Machine learning diagnostic model	Improved prediction for one condition	Single condition, not multi-factorial risk
[3]	Hair fall prediction	Individual classifiers vs. an ensemble	Ensemble outperformed all individual classifiers	Classical models only
[4]	Hair fall prediction	Random Forest with hereditary, hormonal, medical and nutritional factors	Random Forest outperformed XGBoost, CatBoost and LightGBM	Conflicts with [3]; result specific to their dataset
[5]	Hair loss from genetic and scalp data	Classifiers vs. neural networks	Neural networks captured richer interactions	Lower interpretability
[6]	Healthy hair vs. Alopecia Areata	Image features with SVM and KNN	SVM 91.4%, KNN 88.9% accuracy	Image-based and binary; not tabular risk tiers
[12]	Hair loss severity from survey data	Survey-based clinical and lifestyle features	Used ROC-AUC and macro-F1 for class imbalance	Not compared with foundation models

A critical reading of these studies shows four points. First, the conclusions about the best model conflict: an ensemble was best in [3], whereas Random Forest beat several boosting methods in [4]. Because the datasets, targets and metrics differ, the two findings cannot be reconciled, and neither can be assumed to hold for a new dataset. Second, the studies target different outcomes (a single condition, image classes, or severity), so their reported accuracies are not comparable with each other or with the present study. Third, methodological care varies. Only [1] stresses the control of leakage inside cross-validation,

and only [12] explicitly uses metrics that account for class imbalance. Fourth, interpretability is treated unevenly: [5] found that more flexible models were harder to interpret, and among these studies only [1] was designed as an explainable system.

For the present study, these observations lead to three design decisions. The comparison of models is made on the same data, split and metrics, so that any difference can be attributed to the models. The metrics include macro-averaged measures, because the classes are unequal in size. The explanation method is applied to every model, so that interpretability is compared and not assumed.

2.2 Tree ensembles and gradient boosting

A single decision tree is easy to interpret but unstable, because small changes in the training data can change its structure. Ensemble learning combines several simple models to obtain a more accurate and stable prediction. In bagging-based ensembles such as Random Forest [13], many trees are grown on random samples of the data and random subsets of features, and their votes are combined. Random Forest is robust and needs little tuning, which is one reason it appears often in applied studies such as [4].

Boosting builds the ensemble in a different way: models are trained one after another, and each new model corrects the errors of the ensemble built so far. Friedman [14] showed that this can be treated as gradient descent in function space. At step m , a new tree h_m is fitted to the negative gradient of the loss (the pseudo-residuals) and added to the current model with a learning rate v :

$$r_{im} = - \left[\frac{\partial L(y_i, F(x_i))}{\partial F(x_i)} \right]_{F=F_{m-1}} \quad (1)$$

$$F_m(x) = F_{m-1}(x) + v h_m(x) \quad (2)$$

Because each tree targets the remaining error, boosted trees can represent non-linear feature interactions that a single tree cannot, and they have long been the standard method for structured tabular data. XGBoost [15] made this approach scalable by adding a regularised objective, sparsity-aware split finding and efficient system design, and LightGBM [16] increased training speed with histogram-based splits and sampling techniques.

Encoding categorical variables is a recurring difficulty in boosting, since simple encodings can produce sparse features and leak information from the target. Prokhorenkova et al. [7]

proposed CatBoost to address this. It computes target statistics for categorical values using an artificial ordering of the training data, so that the statistic for a record never uses that record's own label, and it applies the same idea to the boosting procedure itself (ordered boosting) to reduce prediction shift. CatBoost also builds symmetric (oblivious) trees, in which the same split condition is used across a whole level, which acts as a regulariser and makes prediction fast. The authors reported better results than other boosting implementations on the benchmarks they used. These properties make CatBoost a strong, well-established baseline for tabular clinical data, which is why it is used as the tuned reference model in this study. The finding of Patel et al. [4] that Random Forest outperformed CatBoost shows, however, that the ranking of tuned tree-based methods depends on the dataset.

2.3 Tree-based models and deep learning on tabular data

Deep learning has been very successful on images and text, but its advantage on tabular data has been questioned. Shwartz-Ziv and Armon [17] compared XGBoost with several recent deep models on datasets that were not used in the deep models' original papers. XGBoost generally performed better on those datasets and needed much less tuning, and an ensemble of the deep models with XGBoost performed better than either alone. Grinsztajn et al. [18] carried out a large benchmark and concluded that tree-based models remained state of the art on medium-sized tabular datasets, even when their speed advantage was ignored, and they discussed properties of tabular data (such as uninformative features and irregular target functions) that suit trees better than standard neural networks.

Tuning is a real cost. A gradient-boosted model has several hyperparameters, and finding good values means training the model many times, usually with cross-validation, and it needs some experience to choose sensible ranges. A method that gives comparable results without this step would save time, and it would also remove one source of unfairness in comparisons, namely that a baseline may be tuned more or less carefully than the new method. In the present study CatBoost was tuned by grid search so that the baseline was not handicapped.

These studies matter for the present work in two ways. They justify the use of a tuned gradient-boosted model as the strong baseline that any new method must be compared with. They also explain why tabular foundation models attracted attention: if a pretrained

model can match tuned trees without a tuning stage, it would remove a step that currently takes time and expertise.

2.4 Transformers, attention and in-context learning

The transformer architecture [19] is built on the attention mechanism. Each element of the input is compared with every other element, and the comparison determines how much each element contributes to the representation of the others. In scaled dot-product attention, the queries Q , keys K and values V are combined as:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (3)$$

where d_k is the dimension of the keys. Attention lets the model relate any element of the input to any other regardless of their distance, and it can be computed for all elements in parallel, which is one reason why transformers can be trained on very large collections of data. Large language models showed that a transformer trained once on a very large collection of data can perform new tasks from examples placed in its input, without any change to its weights. This is called in-context learning, and a model that supports it across many tasks is called a foundation model.

Applied to tabular data, the idea is to pretrain a transformer on a very large number of artificial tabular datasets, so that it learns the general skill of predicting a target from labelled examples. At prediction time, the labelled training rows are given to the model as context together with the query rows, and the prediction is obtained in a single forward pass with no gradient-based training. The training set therefore does not change the model's weights, and it acts instead as input. A consequence is that the cost of a prediction grows with the size of the context, which matters in practice (Section 4.4).

2.5 Tabular foundation models

TabPFN [8] introduced this idea for tabular classification. It was presented as a transformer that solves small tabular classification problems in about a second, without hyperparameter tuning. A later version, published in Nature [20], extended the approach to larger and more varied tables. The authors reported that it outperformed tuned tree-based ensembles on datasets of up to 10,000 samples and 500 features, in much less time. This is the evidence behind the expectation that zero-shot models can compete with tuned boosting on small and medium-sized tables. The dataset used in the present study has 21,606 records, so its

full training partition (17,284 rows) is larger than the range for which that result was reported, which makes the comparison in this study informative about behaviour beyond that range.

TabFM [9], released by Google Research, is a more recent model. It alternates row-wise and column-wise attention over the input table before an in-context transformer produces the predictions, whereas TabPFN applies attention mainly across samples. On the TabArena benchmark suite it was reported to outperform heavily tuned gradient-boosted baselines in a purely zero-shot setting [9]. An independent evaluation by a third party [10], run on three machines and thirteen datasets, confirmed its competitive zero-shot accuracy against XGBoost, Random Forest and TabPFN, but it also documented four defects in the official implementation. The evidence reviewed here on TabFM comes from general benchmarks, not from health or dermatology data, and it comes from a company blog and a public repository, not from peer-reviewed publication. It should therefore be treated as provisional, which is one reason why testing TabFM independently on a new problem is useful.

The two foundation models differ in design, and the difference matters for how they are used. In TabPFN, attention is applied mainly across the samples of the table, so each query row is related to the labelled rows in the context. In TabFM, attention is applied alternately across rows and across columns before the in-context transformer reads the compressed representation of each row. Both approaches make the training data part of the input at prediction time. This has three consequences for a practical comparison. First, the amount of context that can be used is limited by memory and time, and a large training set may have to be reduced to a smaller context. Second, the cost of each prediction depends on the context size and not only on the model, which is unlike a boosted-tree model whose prediction cost is small once it is trained. Third, the training data must be available whenever predictions are made, which is a consideration for deployment. These points are examined for hair fall data in Chapter 4.

2.6 Explainability with SHAP

Boosted ensembles and attention-based models both give no direct explanation for an individual prediction. In clinical use this is a serious limitation, because a practitioner needs a reason for a risk estimate before acting on it. Shapley values, introduced in cooperative game theory [21], distribute the payoff of a game fairly among its players

according to their contributions. Lundberg and Lee [11] applied this idea to prediction. In their SHAP framework each feature is a player and the model output is the payoff, and the prediction for an instance is split into additive contributions: $M f(x) = \varphi_0 + \sum_{j=1}^M \varphi_j$ (4)

$$f(x) = \varphi_0 + \sum_{j=1}^M \varphi_j \quad (4)$$

where φ_0 is the expected model output and φ_j is the contribution of feature j . The Shapley value of feature j averages its marginal contribution over all subsets S of the other features F :

$$\varphi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!(|F| - |S| - 1)!}{|F|!} [f_{S \cup \{j\}}(x) - f_S(x)] \quad (5)$$

Lundberg and Lee [11] showed that this is the only additive attribution that satisfies the properties of local accuracy, missingness and consistency. Computing Eq. (5) exactly requires evaluating the model for a number of feature subsets that grows exponentially with the number of features, so practical algorithms are needed. For tree models, TreeExplainer [22] computes exact values in polynomial time by using the tree structure. For models without such structure, Kernel SHAP [11] estimates the values by querying the model on perturbed inputs and fitting a weighted linear regression. It works for any model, but the number of model calls grows with the number of explained instances and the number of samples per instance, which is costly when each call is expensive, as with an in-context transformer.

The results of a SHAP analysis are usually shown in three ways. A global bar plot ranks features by their mean absolute contribution. A beeswarm plot shows, for every explained record, the contribution of each feature and colours it by the feature's value, so that the direction of the effect can be read (for example, whether high values of a feature raise or lower the prediction). A waterfall plot shows how the contributions of the features of one record move the prediction from the baseline to its final value. The bar plot and the beeswarm plot are used in Chapter 4.

SHAP supports both a global view (mean absolute values give an importance ranking across the dataset) and a local view (the contribution of each feature to one patient's prediction). SHAP explains what a model has learned, and it does not show that the relationships are causal. The same method can be applied to different model types, which is why it was chosen here. Two cautions apply in practice. An explanation is only as

reliable as the model, so a model that has learned a spurious pattern will give an explanation that reflects the pattern. In addition, Kernel SHAP values are estimates: they depend on the number of samples per instance and on the background data, and when only a few instances are explained the ranking of features with similar importance can change from one run to another. This is why the analysis in Chapter 4 relies on the shared set of leading features and not on the exact order.

2.7 Evaluating and comparing classifiers

Accuracy, the proportion of correct predictions, can be misleading when classes differ in size, because a model that favours the largest class can still score well. Sokolova and Lapalme [23] analysed performance measures for classification and described macro-averaging, in which a measure such as precision, recall or the F1-score is computed for each class and then averaged with equal weight, so that small classes count as much as large ones. For multi-class problems, ROC-AUC can be computed by treating each class in turn against all the others and averaging the results.

Reporting differences between two classifiers also requires a statistical test. McNemar's test [24] compares two classifiers evaluated on the same test records. It uses only the records on which the two classifiers disagree: b records that the first classified correctly and the second did not, and c records for the reverse. Under the null hypothesis that both make errors at the same rate, b and c are expected to be equal. Dietterich [25] compared several tests for supervised learning algorithms and found that McNemar's test has a low rate of false positive results and is a suitable choice when each algorithm can be trained and run only once, as was the case here. The test has limits. It does not measure variation from the choice of training set or from random seeds, and a non-significant result does not show that the classifiers are equivalent, only that no difference was detected.

Model selection needs care as well. Hyperparameters such as the depth of a tree should be chosen on data that is not used for the final test, and k -fold cross-validation does this by repeatedly training on part of the training data and scoring on the remaining part. Stratified folds keep the class proportions in every fold, which matters when classes are unequal. Selecting hyperparameters on the test data would give an optimistic estimate of performance, which is the kind of leakage that Widowati et al. [1] warned about.

The risk tiers in this study are ordered (Low, Moderate, High), and standard multi-class classifiers treat them as unrelated labels. Ordinal classification methods take the order into

account. Frank and Hall [26], for example, proposed a simple approach that turns an ordinal problem with k classes into $k-1$ binary problems, each asking whether the target is greater than a given tier, so that any binary classifier can be used. Such methods may improve results for the middle tier of an ordered scale. They were not part of this study, whose aim was to compare three methods on the standard multi-class task, and they are noted as further work (Section 5.3).

2.8 Critical summary and research gap

Table 2.2 brings the reviewed methods together.

Table 2.2: Model families relevant to this study

Model family	Representative studies	Strengths	Known limitations	Role in this study
Random Forest and other classical classifiers	[13], [3], [4]	Robust, little tuning	Ranking versus boosting varies by dataset	Not evaluated (context only)
Gradient-boosted trees	[14], [15], [16], [7]	Strong on tabular data; tuned baseline	Needs tuning; needs enough training data	CatBoost, tuned baseline
Neural networks	[5], [17], [18]	Can capture complex interactions	Often no better than trees on tabular data; harder to interpret	Not evaluated (context only)
Tabular foundation models	[8], [20], [9], [10]	No tuning; strong with small data	Cost grows with context; TabFM evidence provisional	TabPFN and TabFM, zero-shot
Post-hoc explanation	[21], [11], [22]	Model-agnostic feature attribution	Kernel estimation is costly for large-context models	SHAP for all three models

Three points follow from Table 2.2. Tuned gradient boosting is the method that any new tabular method must be compared with, so a fair benchmark needs a properly tuned CatBoost and not a default configuration. Tabular foundation models are promising mainly because they remove the tuning step and work with little data, so a fair benchmark also needs a varied training size and not only the full dataset. Finally, since both kinds of model are opaque, a fair benchmark for health data should compare what each model relies on, and not only how accurate it is. The design of the present study follows these three points (Chapter 3).

A further point concerns the data. A single company dataset, as in this study, has feature-target relationships that are fixed by how the data were produced. A model trained on such a dataset can be expected to recover those relationships, and how well it does so is informative about the model. It says little about how the model would perform on patients from a real clinic, where the relationships are noisier and are not known in advance. For

this reason the study is presented as a benchmark of model behaviour and not as a clinical validation.

The reviewed work shows the following.

1. Studies on hair loss prediction have used classical classifiers, ensembles, Random Forest, neural networks and image features [1]–[6], [12]. Their conclusions about the best model conflict ([3] versus [4]), and their targets differ, so the results are not comparable.
2. Interpretability remains a concern. The neural models in [5] were harder to interpret, and among the reviewed hair loss studies only [1] was designed as an explainable system.
3. Tuned gradient boosting is an established baseline for tabular data [17], [18], and CatBoost [7] is a leading implementation, but it has not been compared with tabular foundation models on hair fall data.
4. TabPFN [8], [20] and TabFM [9] have been evaluated only on general benchmarks. No reviewed study evaluates them on health survey data, compares the two models with each other and with a tuned gradient-boosted model, or applies a common explanation method to all of them. TabFM's reliability is also only partly established [10].
5. The behaviour of these models when training data is limited, which is common in health settings, has not been examined for hair fall risk, and neither has the cost of explaining large-context models with model-agnostic SHAP. This study addresses these gaps by benchmarking CatBoost, TabPFN and TabFM on multi-tier hair fall risk classification with the same data, split and metrics, by varying the training context size, and by applying SHAP to all three models.

CHAPTER 3

METHODOLOGY

This chapter describes how the study was conducted to meet the research objectives stated in Section 1.4. It covers the research design and approach, the study area and population, sample selection and size, the data collection, the data analysis approach and tools, and the ethical considerations. Table 3.1 links each method to the objective it serves.

Table 3.1: Link between the research objectives and the methods used

Specific objective (Section 1.4)	Method used	Section
1. Compare predictive performance and test significance	Common train/test split; CatBoost tuning; TabPFN and TabFM inference; macro-averaged metrics, ROC-AUC, ordinal-aware measures; McNemar's test at every training size	3.5, 3.8.3, 3.8.4, 3.8.7
2. Evaluate the effect of training context size	Every model trained on, or supplied with, 500, 2,000 and all 17,284 training rows and scored on the same 4,322 test records	3.8.5
3. Explain predictions with SHAP	Descriptive analysis of the data; TreeExplainer for CatBoost, KernelExplainer for TabPFN and TabFM, at every training size	3.8.2, 3.8.6
4. Assess computational cost and implementation limitations	Recording fit and prediction times; documenting memory and software problems and their workarounds	3.8.8

3.1 Research design

The study used a quantitative, experimental design based on secondary data. It was a model-comparison (benchmarking) study: the same dataset, the same train/test partition and the same evaluation measures were applied to three models at three training sizes, so that differences in results could be attributed to the models and to the amount of training data, and not to the data. No primary data were collected from people, so there was no field study area and no direct contact with respondents.

The study followed the pipeline in Fig. 3.1. The data were validated, cleaned and split once. CatBoost was tuned by cross-validation and trained on the training rows, while TabPFN and TabFM received the same rows as context without any training. All nine model runs (three models at three sizes) were evaluated on the same held-out test partition, compared with a statistical test at each size, and explained with SHAP. Each stage of Fig.

3.1 was implemented as one numbered step (Steps 1 to 10) in a locally developed browser-based notebook, the Hairfall Lab (Section 3.8.9), which stores the code, printed output, figures and result files of every run.

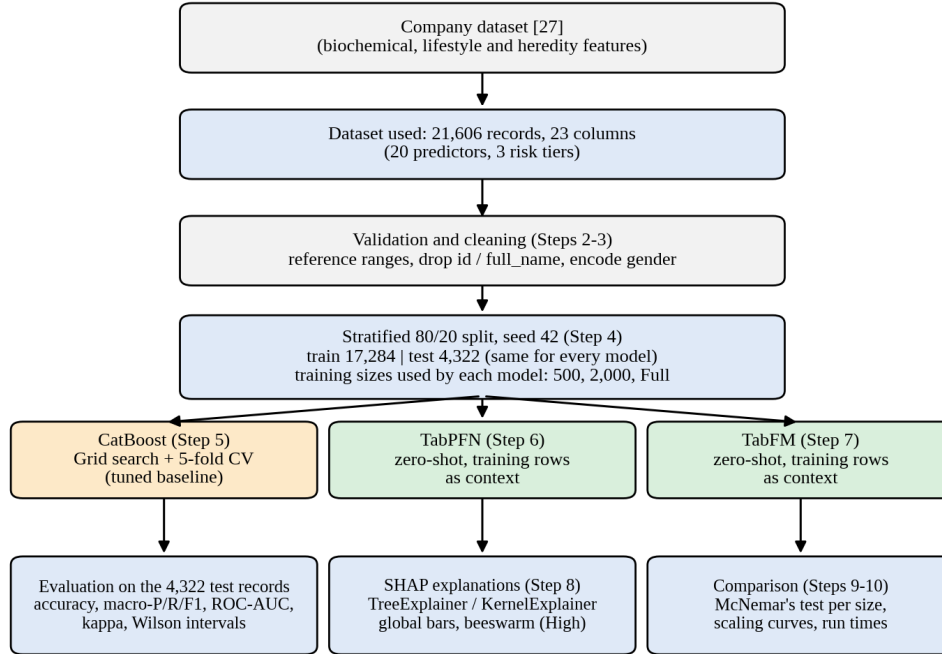


Fig. 3.1: Block diagram of the research methodology

The design was chosen because the research questions ask for a like-for-like comparison of models on one task. An experimental design is appropriate when the aim is to compare methods under controlled conditions, and a benchmarking study fits the objectives because none of them requires new data collection: each asks how existing methods behave on a defined task. A single fixed split and a fixed random seed (42) made the results reproducible. Each stage of Fig. 3.1 is linked to an objective in Table 3.1, so that no step was included without a purpose in the research.

3.2 Research approach

A quantitative, deductive approach was used. The proposition examined was that zero-shot tabular foundation models can match a tuned gradient-boosted model on multi-tier hair fall risk classification, and that the size of the training data changes the comparison. It was examined with numerical performance measures and statistical tests, and the predictions were then explained with SHAP. A quantitative approach was suitable because the research questions ask about measurable differences in performance, data efficiency, feature contribution and cost.

3.3 Study area

The study was not a field study, so there was no geographical study area. The data were taken from a company dataset [27]. The model runs were executed in Kaggle Notebooks on a cloud GPU, and the analysis of the results was done on a local computer (Section 3.8.9).

3.4 Study population

The population was the set of patient records describing demographic, biochemical, clinical and lifestyle attributes related to hair fall. The dataset had 21,606 records with 23 columns: an identifier (id), a name (full_name), 20 predictor variables and the target variable hair_fall. There were no duplicate rows and no missing values. The target had three ordinal tiers: 0 (Low, 9,723 records, 45.0%), 1 (Moderate, 7,562 records, 35.0%) and 2 (High, 4,321 records, 20.0%). The classes were therefore unequal in size, which is the reason for the macro-averaged measures in Section 3.8.7.

3.5 Sample selection

The whole dataset was used, so no separate sampling of records was done. It was divided into a training and a test partition by a stratified 80/20 split with a fixed random seed of 42, so that class proportions were preserved in both parts. The test partition was held out and was not used to tune any model. The same partition was used for all three models at all training sizes.

For the training-size experiment (Section 3.8.5), the training subsets of 500 and 2,000 rows were drawn from the training partition by stratified random sampling (seed 42), so that every subset kept the 45%, 35% and 20% class shares. The third size, Full, was the whole training partition of 17,284 rows. The same subset was given to all three models at a given size.

3.6 Sample size

The full dataset had 21,606 records. The split gave 17,284 training records and 4,322 test records (1,945 Low, 1,513 Moderate and 864 High). Each model was run with 500, 2,000 and 17,284 training rows (about 2.9%, 11.6% and 100% of the training partition), which gives nine model runs. All nine were evaluated on all 4,322 test records, so no run used a reduced test set.

3.7 Methods of data collection

No primary data were collected from people. The study used secondary data and computational modelling and measurement.

The dataset was provided by a company [27]. It contains physiological biomarker data, which are continuous clinical indicators relevant to hair health, together with demographic, lifestyle and hereditary risk factors. The respondent names in the dataset are stored only in encoded form and were not used. It is a company dataset and not a clinical registry. The consequences for the interpretation of the results are discussed in Chapter 4 and Chapter 5.

The 20 predictor variables fell into four groups (Table 3.2).

Table 3.2: Predictor variables

Group	Variables
Demographics and heredity	age (years); gender (Female, Male, Other); family_hair_fall_history (0/1)
Serum and physiological biomarkers	total_protein (g/dL); iron ($\mu\text{g/dL}$); calcium (mg/dL); vitamin_d (ng/mL); manganese ($\mu\text{g/L}$); alt_liver (U/L); body_water_content (%); stress_level (0–40, modelled on the Perceived Stress Scale)
Hair condition scores	total_keratine and hair_texture (composite 0–100 scores, not standard laboratory measurements)
Clinical and lifestyle flags (0/1)	chronic_illness, anemia, stress, late_night_sleep, sleep_disturbance, water_reason (exposure to hard or chemically treated water), chemical_use (hair dyes, relaxers and other chemical treatments)

Table 3.3 gives the range of the continuous predictors in the full dataset.

Table 3.3: Descriptive statistics of the continuous predictors (all 21,606 records)

Predictor	Mean	Standard deviation	Minimum	Maximum
age (years)	32.83	9.67	20.00	55.00
total_protein (g/dL)	7.20	0.60	4.80	9.50
calcium (mg/dL)	9.30	0.60	7.00	11.50
iron ($\mu\text{g/dL}$)	98.47	30.94	10.00	217.00
vitamin_d (ng/mL)	28.11	11.75	4.00	78.40
alt_liver (U/L)	25.66	13.78	5.00	81.00
manganese ($\mu\text{g/L}$)	8.02	2.98	1.00	20.00
body_water_content (%)	54.98	7.89	35.00	75.00
stress_level (0–40)	18.03	7.90	0.00	40.00
total_keratine (0–100)	49.88	19.80	0.00	100.00
hair_texture (0–100)	50.16	19.67	0.00	100.00

The predictors cover plausible ranges for adults aged 20 to 55 years. The two hair-condition scores span the whole 0 to 100 scale with means near 50, and the stress score spans the whole 0 to 40 scale. The gender attribute had 12,109 male, 9,485 female and 12 other records, so the "other" group was too small to be analysed separately.

Model outputs (predictions, class probabilities, SHAP values) and running times were collected by running the experiments in Section 3.8, which is the measurement part of the study.

3.8 Data analysis approach and tools

The data were analysed in the steps below. The analysis was done in Python, and the tools are listed in Section 3.8.9. The code of every step is given in Appendix A.

3.8.1 Data preprocessing

The following steps were applied before modelling (Steps 2 and 3 of the pipeline).

1. **Validation against clinical reference ranges.** Seven biomarkers were compared with a normal range and with limits of physiological impossibility (Table 3.4). Abnormal but possible values were kept, because they carry the signal that predicts hair fall. Only impossible values would have been removed, and none was found.
2. **Identifier removal.** The columns `id` and `full_name` carry no predictive information and could create false associations, so they were removed.
3. **Encoding.** The gender attribute was mapped to numbers (0 = Female, 1 = Male, 2 = Other). The target was already coded as 0, 1 and 2. All other predictors were numeric or binary and were used as they were. TabPFN and TabFM received these raw values without scaling.
4. **Missing values and duplicates.** The data were checked and no missing values or duplicate rows were found, so no imputation was needed.

Table 3.4: Biomarkers compared with clinical reference ranges (all 21,606 records)

Biomarker	Normal range	Below normal	Above normal	Impossible values	Outside normal range (%)
total_protein (g/dL)	6.0–8.3	406	605	0	4.7
calcium (mg/dL)	8.5–10.5	1,675	392	0	9.6
iron (µg/dL)	60–170	2,217	209	0	11.2
vitamin_d (ng/mL)	25–80	8,606	0	0	39.8
alt_liver (U/L)	7–56	2,300	403	0	12.5
manganese (µg/L)	4–15	1,960	208	0	10.0
body_water_content (%)	45–65	2,225	2,227	0	20.6

No fitted transformation (such as scaling or target encoding) was applied, so the warning of Widowati et al. [1] about leakage from preprocessing did not apply to the modelling stage: nothing was learned from the test records before evaluation.

3.8.2 Descriptive analysis of the dataset

Before interpreting the models, the relationship between each predictor and the risk tier was described in the full dataset. For each continuous predictor, the mean was calculated in each risk tier, and for the binary flags the share of records with the flag was calculated in each tier. For all predictors, Spearman's rank correlation with the ordinal risk tier was calculated. This analysis was done with the pandas and SciPy packages. It was not used to select features or to train any model. Its purpose was to provide an independent reference against which the SHAP explanations could be compared (Section 4.3).

3.8.3 Models and justification

Three models were selected to represent three different approaches to tabular prediction, in line with Objective 1.

3.8.3.1 CatBoost

CatBoost [7] is a gradient-boosted decision tree method that builds trees one after another, each correcting the errors of the ensemble so far, as described in Section 2.2. At iteration t it minimises a regularised objective:

$$L^{(t)} = \sum_i L(y_i, F_{t-1}(x_i) + h_t(x_i)) + \Omega(h_t) \quad (6)$$

where L is the multi-class cross-entropy loss, F_{t-1} is the ensemble built so far, h_t is the new tree and Ω penalises tree complexity. CatBoost reduces target leakage through ordered target statistics and builds symmetric (oblivious) trees, which limits overfitting. It was selected as the tuned, well-established baseline whose performance the foundation models had to match. Grid search was used to tune two hyperparameters that control model complexity and learning speed: the tree depth (deeper trees model more interactions but overfit more easily) and the learning rate (smaller values need more trees).

3.8.3.2 TabPFN

TabPFN [8], [20] is a transformer pretrained offline on synthetic datasets. It does not train on the target data. It receives the labelled training rows D_{train} and a query x_{test} together and returns the predictive distribution in a single forward pass:

$$P(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}) = \int P(y_{\text{test}} \mid x_{\text{test}}, \theta) P(\theta \mid D_{\text{train}}) d\theta \quad (7)$$

This is done with the self-attention mechanism of Eq. (3). TabPFN was selected as the established tabular foundation model, which needs no hyperparameter tuning or feature scaling.

3.8.3.3 TabFM

TabFM [9] is a tabular foundation model released by Google Research. It alternates row-wise and column-wise attention over the table before an in-context transformer produces the prediction, and it also needs no training or tuning:

$$P(y_{\text{test}} \mid x_{\text{test}}, D_{\text{train}}) = \text{TabFM}(x_{\text{test}}, D_{\text{train}}) \quad (8)$$

TabFM was selected as a second, independently developed foundation model, so that the study could test whether the behaviour of zero-shot models holds for more than one architecture.

3.8.4 Experimental setup

CatBoost. For each training size, a grid search was run over tree depth $\{4, 6, 8\}$ and learning rate $\{0.01, 0.03, 0.1\}$. Each of the nine combinations was scored by stratified 5-fold cross-validation on the training rows of that size, using macro-F1. Within each fold, up to 1,000 iterations were allowed with early stopping after 50 rounds monitored on the validation fold of the training data, and the number of iterations of a combination was the mean of the best iteration over the five folds. All models used the MultiClass loss,

$l2_leaf_reg = 3$ and a random seed of 42. The combination with the highest mean macro-F1 was chosen, and the final model was trained on all rows of that size with the chosen depth, learning rate and iteration count, with no further early stopping. The test partition was therefore not used at any point to tune or to stop training.

TabPFN. The default configuration of the TabPFN package was run on a GPU with a random seed of 42 and the option that lifts the package's pretraining size limits. The training rows of each size were supplied as context, and no tuning was done.

TabFM. The PyTorch implementation of TabFM version 1.0.1 was used with a single ensemble member ($n_estimators = 1$), on a GPU. The training rows of each size were supplied as context, and no tuning was done. Because the full 17,284-row context exceeded the memory of the GPU with the default attention implementation, the attention was evaluated in chunks of 1,024 queries and the column chunk size was set to 4. These settings change the memory use and not the computation (Section 3.8.8).

The settings of the three models are summarised in Table 3.5.

Table 3.5: Settings of the three models

Setting	CatBoost	TabPFN	TabFM
Tuning	Grid search over depth and learning rate; 5-fold stratified CV; macro-F1	None	None
Training data used	500, 2,000 and 17,284 rows (fitted)	500, 2,000 and 17,284 rows (context)	500, 2,000 and 17,284 rows (context)
Main parameters	MultiClass loss, $l2_leaf_reg$ 3, up to 1,000 iterations with early stopping (50 rounds) inside CV; final iterations from CV	Package defaults; pretraining limits lifted	Version 1.0.1, PyTorch backend, $n_estimators = 1$
Hardware	CPU	GPU (NVIDIA Tesla T4)	GPU (NVIDIA Tesla T4)
Random seed	42	42	42 (context sampling and package default)

The choices in the setup have reasons. The split was stratified so that the three tiers had the same proportions in the training and test partitions. Macro-F1 was used to select the CatBoost setting because it gives each tier equal weight, which is consistent with the macro-averaged measures used for evaluation. Five folds were used as a compromise between the reliability of the score and the computing time. Early stopping ends training when the score has not improved for 50 rounds, which limits overfitting. The grid was small (nine combinations) because it covered the two parameters that most affect complexity and speed and kept the search within the available computing time.

The tuning effort was deliberately unequal: CatBoost was tuned by grid search, while TabPFN and TabFM were not tuned. This asymmetry is central to the research question, which asks whether models that need no tuning can match one that does.

3.8.5 Training size experiment

To meet Objective 2, all three models were run with training sizes of 500, 2,000 and 17,284 (Full) rows. CatBoost was tuned and fitted separately at each size, and TabPFN and TabFM received the rows of each size as context. Every run was evaluated on the full test partition of 4,322 records, so the nine results are directly comparable, and the three models at each size used exactly the same rows.

3.8.6 Explainability analysis

To meet Objective 3, post-hoc explanations were produced with SHAP [11], whose additive form is given in Eq. (4) and whose Shapley values are defined in Eq. (5). An explanation was produced for every model at every training size, which gives nine analyses.

For CatBoost, TreeExplainer [22] was used, which computes exact Shapley values from the tree structure, on 1,000 test records drawn at random (seed 42). TabPFN and TabFM have no tree structure, so KernelExplainer was used, which estimates the values by querying the model repeatedly. Because each query passes through the full in-context mechanism, these analyses used a random subset of the test partition (seed 42): 30 records for TabPFN and 20 for TabFM, 100 kernel evaluations per explained record, and a background sample summarised by k-means with 10 centres for TabPFN and 5 for TabFM. A wrapper padded single-row queries, because some models fail on one row. The High risk class was chosen for the beeswarm plots because it is the tier of greatest clinical interest and because a single class is needed to show the direction of an effect.

For each analysis, the mean absolute SHAP value of every feature was computed for each class and averaged over the three classes to give a global importance, and the features were ranked by it. Global importance bar plots and beeswarm plots for the High risk class were produced. Features were compared across models by rank and not by the size of the values, since the numbers of explained records differed.

3.8.7 Evaluation metrics and statistical testing

To meet Objective 1, predictions were evaluated on the test partition with macro-averaged measures [23], which give the three classes equal weight. TP_k , FP_k and FN_k are the true positives, false positives and false negatives of class k , and N is the number of test records.

$$Accuracy = \frac{\sum_{k=0}^2 TP_k}{N} \quad (9)$$

$$Macro - Precision = \frac{1}{3} \sum_{k=0}^2 \frac{TP_k}{TP_k + FP_k} \quad (10)$$

$$Macro - Recall = \frac{1}{3} \sum_{k=0}^2 \frac{TP_k}{TP_k + FN_k} \quad (11)$$

$$Macro - F1 = \frac{1}{3} \sum_{k=0}^2 \frac{2 \cdot Precision_k \cdot Recall_k}{Precision_k + Recall_k} \quad (12)$$

Because the tiers are ordered, four further measures were calculated from the confusion matrices. Cohen's kappa [28] measures the agreement between the predicted and true tiers beyond the agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (13)$$

where p_o is the observed share of correct predictions and p_e is the share expected by chance from the row and column totals of the confusion matrix. Quadratic weighted kappa [29] uses the same idea but penalises an error by the squared distance between the true and predicted tiers, so that confusing Low with High counts four times as much as confusing neighbouring tiers. The share of predictions within one tier of the true tier and the mean absolute error, in tiers,

$$MAE = \frac{1}{N} \sum_{i=1}^N |\hat{y}_i - y_i| \quad (14)$$

describe how far the errors were from the true tier.

A confusion matrix counts, for each true tier, how many records were assigned to each predicted tier. Precision for a tier is the share of records predicted as that tier that truly belong to it, and recall is the share of records of that tier that were found. The F1-score combines the two into one value that is high only when both are high. ROC-AUC was computed with a one-vs-rest strategy and macro-averaged. Per-class F1-scores and

confusion matrices were also produced. Macro-averaging was used because the classes were unequal in size (Section 3.4).

Pairwise differences in accuracy between the models were tested with McNemar's test [24] with continuity correction on the same test records, using the counts b and c of records that one model classified correctly and the other did not:

$$\chi^2 = \frac{(|b - c| - 1)^2}{b + c} \quad (15)$$

The test was applied to the three pairs of models at each of the three training sizes, which gives nine tests. The uncertainty of each accuracy value that comes from the finite size of the test sample was described with 95% Wilson score confidence intervals [30], calculated from the accuracy and the number of test records. A difference was called significant at $\alpha = 0.05$. McNemar's test was chosen because each model was trained and run once on one split, a situation for which it has a low false-positive rate [25].

3.8.8 Computational assessment

To meet Objective 4, the running time of each model was recorded during training or context fitting and during prediction, and the practical problems met when running the models (memory limits, software errors and the workarounds used) were documented.

Running times were measured as wall-clock time with a timer around the fitting call and around the prediction call. Predictions for the test partition were made in batches (500 records for TabPFN and 100 for TabFM), and the results were written to a checkpoint file after each batch, so that a run interrupted by a session limit could be resumed. The Kaggle runs also halved the batch size automatically after a GPU out-of-memory error. Because the runs were made on shared cloud hardware, the times are approximate and can vary between sessions, and they are used to show how the cost changes with the context size and not to give exact benchmarks. One run, TabFM with 500 rows, was resumed from a checkpoint, so its prediction time was not measured (Section 4.4).

3.8.9 Tools and environment

The experiments were written in Python. The final runs of the models and of the SHAP analyses were executed in Kaggle Notebooks on an NVIDIA Tesla T4 GPU (Table 3.6). The code was developed and stored in the Hairfall Lab, a browser-based notebook written for this study, in which the steps are run one at a time in a persistent Python kernel. Every

run is saved with its code, printed output, figures and result files in a database and an object store, and in a folder per step. The Lab runs on a local Ubuntu computer (eight CPU cores, 16 GB of RAM, no GPU), where the first runs and the development were done. The steps run on Kaggle are packaged as notebook files, and their result files are imported back into the Lab so that all results are kept in one place.

Data handling used pandas and NumPy, and scikit-learn [31] was used for the split, cross-validation and metrics. The models came from the catboost, tabPFN and tabfm (PyTorch backend) packages, the explanations from the shap package, the significance test from statsmodels, and the plots from matplotlib.

Table 3.6: Experimental environment

Item	Specification
Platform of the final model and SHAP runs	Kaggle Notebooks (cloud)
GPU	NVIDIA Tesla T4 (TabPFN, TabFM and the SHAP analyses of both); CatBoost ran on the CPU
Memory	33.7 GB RAM
Operating system	Linux (glibc 2.35)
Language	Python 3.12.13
Main libraries	CatBoost 1.2.10, TabFM 1.0.1 (PyTorch), scikit-learn 1.6.1, pandas 2.3.3, NumPy 2.0.2, statsmodels 0.14.6; TabPFN and SHAP were installed at run time without a fixed version (version 9.0.0 and 0.52.0 on the local computer)
Development computer	Ubuntu Linux, 8 CPU cores, 16 GB RAM, no GPU (Hairfall Lab)

3.9 Ethical considerations

The study used only secondary data from a company dataset [27]. No participants were recruited, no interviews or surveys were carried out, and no intervention was made, so ethical approval for data collection was not required. The columns id and full_name were dropped before modelling, and no attempt was made to identify any individual. The name column of the dataset holds encoded strings, and it was dropped in the cleaning step before any model was built. The dataset is a single company dataset. It is described as such throughout the thesis, and the results are presented as a benchmark of model behaviour. They are not presented as a clinical finding or as a diagnostic tool. Model outputs must not be used to make decisions about real patients without validation on real clinical data and review by a qualified clinician.

3.10 Validity, reliability and reproducibility

Several steps were taken to make the comparison valid and the results reproducible. To protect the validity of the comparison, all three models used the same records, the same split and the same measures. The test partition was not used for hyperparameter tuning or for early stopping (Section 3.8.4), and no fitted transformation was applied to the data, so information from the test records did not flow into the models through preprocessing.

To protect reliability, random seeds were fixed at 42 wherever a random choice was made in the modelling: the split, the cross-validation folds, the training subsets, the samples of test records used for SHAP and the model initialisation. The result of a single seed and a single split cannot show how much the numbers vary from run to run, and this is acknowledged as a limitation in Section 5.2. The reruns of the same step in the Lab gave results that differed slightly in the third decimal place (for example, CatBoost with 500 rows scored 0.6976 in an earlier run on a local CPU and 0.6937 in the final run), which shows the size of the variation that comes from the computing platform and library versions.

To support reproducibility, the code is given in Appendix A, the software packages are listed in Section 3.8.9, and the dataset is described in Section 3.7. Some parts cannot be reproduced exactly. TabPFN is used through a package that needs a licence token, its version and that of SHAP were not fixed in the Kaggle runs, and TabFM version 1.0.1 may change in later releases.

Because the dataset came from a single company, the study has no external validity for real patients until it is repeated on clinical data. Its internal validity, meaning whether the comparison between the models is fair, rests on the steps described above.

CHAPTER 4

RESULTS AND DISCUSSION

This chapter presents the results in the order of the research objectives. Section 4.1 compares the predictive performance of the three models, Section 4.2 shows the effect of the training size, Section 4.3 explains the predictions with SHAP, and Section 4.4 reports the computational cost and the implementation problems. Section 4.5 discusses the findings. All values are for the same 4,322 test records unless stated otherwise, and all figures and tables were produced from the saved output of the pipeline steps.

4.1 Predictive performance of the three models

4.1.1 Overall performance

Table 4.1 gives the performance of the nine model runs on the test partition. With the full training set (17,284 rows) the three models reached almost the same accuracy: 78.20% for CatBoost, 78.62% for TabPFN and 78.67% for TabFM. The macro-F1 values were 0.7744, 0.7785 and 0.7802, and the ROC-AUC values were 0.9197, 0.9235 and 0.9233. The two foundation models were ahead of CatBoost by 0.42 and 0.47 percentage points, and TabPFN and TabFM differed by 0.05 points.

Table 4.1: Performance of the three models on the test partition (4,322 records) at each training size

Training rows	Model	Accuracy	Macro-precision	Macro-recall	Macro-F1	Weighted F1	ROC-AUC
500	CatBoost	0.6937	0.6872	0.6537	0.6648	0.6891	0.8611
500	TabPFN	0.7689	0.7686	0.7562	0.7616	0.7699	0.9159
500	TabFM	0.7684	0.7663	0.7555	0.7603	0.7693	0.9151
2,000	CatBoost	0.7497	0.7579	0.7253	0.7377	0.7496	0.9029
2,000	TabPFN	0.7827	0.7854	0.7691	0.7757	0.7844	0.9220
2,000	TabFM	0.7844	0.7881	0.7713	0.7779	0.7863	0.9209
17,284	CatBoost	0.7820	0.7822	0.7686	0.7744	0.7832	0.9197
17,284	TabPFN	0.7862	0.7878	0.7719	0.7785	0.7875	0.9235
17,284	TabFM	0.7867	0.7898	0.7735	0.7802	0.7882	0.9233

The picture changes when the training data are limited. With 500 rows the accuracy was 69.37% for CatBoost, 76.89% for TabPFN and 76.84% for TabFM, so both foundation models were about 7.5 points ahead. With 2,000 rows the values were 74.97%, 78.27% and 78.44%, a lead of about 3.4 points. The lead of the foundation models therefore shrank as the training size grew and was less than half a point with all the data. Whether these differences are larger than chance is tested in Section 4.1.3.

The CatBoost baseline was tuned fairly. Table 4.2 gives the combination chosen by cross-validation at each size. Depth 4, the shallowest depth in the grid, was chosen every time, and the learning rate was 0.03 for 500 and 2,000 rows and 0.1 for the full data. The cross-validated macro-F1 (0.6589, 0.7403 and 0.7669) was close to the macro-F1 on the test partition (0.6648, 0.7377 and 0.7744), which suggests that the tuning did not overfit the training data and that the test partition gave an honest estimate.

Table 4.2: Hyperparameters chosen for CatBoost by cross-validation

Training rows	Depth	Learning rate	Iterations	CV macro-F1	Test macro-F1
500	4	0.03	654	0.6589	0.6648
2,000	4	0.03	999	0.7403	0.7377
17,284	4	0.1	807	0.7669	0.7744

4.1.2 Per-class performance and confusion matrices

Table 4.3 gives the precision, recall and F1-score of each risk tier for the three models with the full training data. Results for the smaller training sizes are given in Appendix C.

Table 4.3: Precision, recall and F1-score by risk tier (full training data)

Model	Tier	Precision	Recall	F1-score	Records
CatBoost	Low	0.8597	0.8478	0.8537	1,945
	Moderate	0.6767	0.7264	0.7007	1,513
	High	0.8103	0.7315	0.7689	864
TabPFN	Low	0.8649	0.8488	0.8568	1,945
	Moderate	0.6796	0.7389	0.7080	1,513
	High	0.8190	0.7280	0.7708	864
TabFM	Low	0.8655	0.8437	0.8545	1,945
	Moderate	0.6787	0.7442	0.7100	1,513
	High	0.8253	0.7326	0.7762	864

All three models classified the Low tier best (F1 between 0.85 and 0.86) and the Moderate tier worst (F1 between 0.70 and 0.71). The High tier had high precision but lower recall (recall between 0.73 and 0.73), which means that the models were cautious about assigning the highest risk and that some High-risk records were placed in the Moderate tier. Fig. 4.1 shows the confusion matrices.

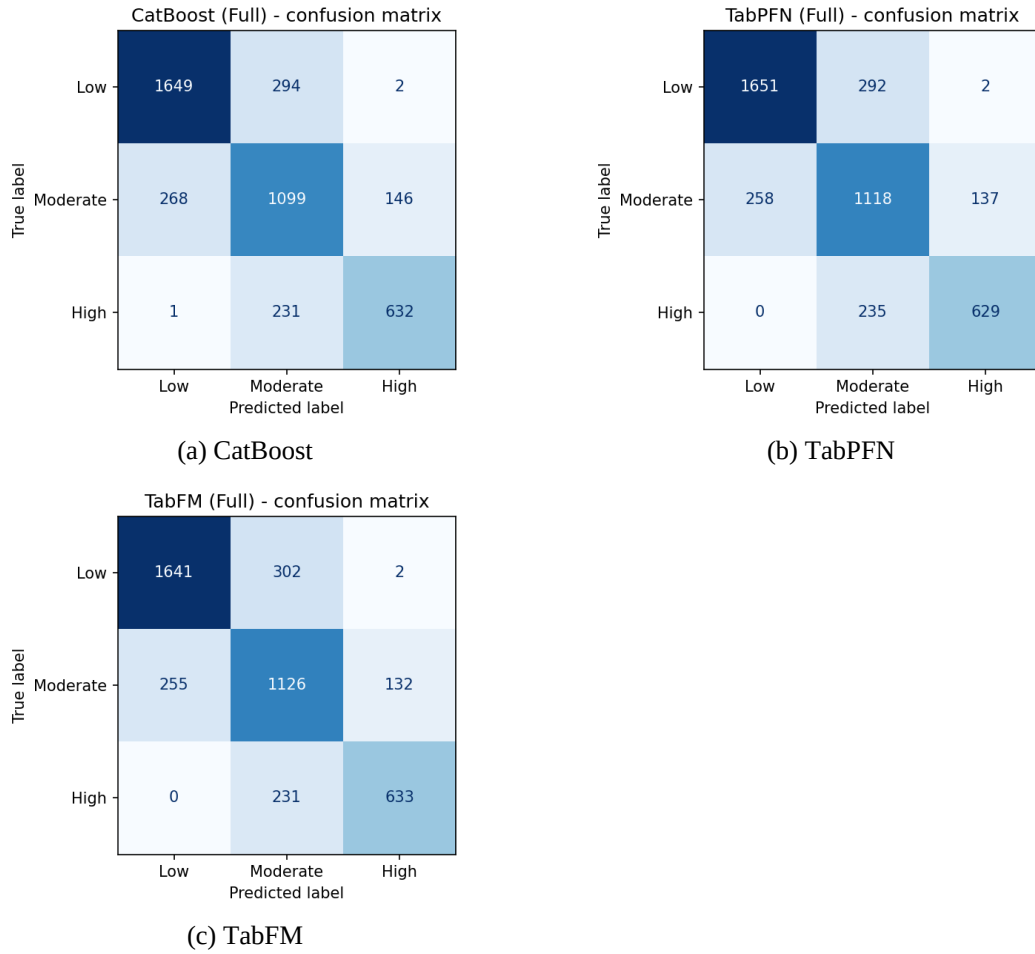


Fig. 4.1: Confusion matrices of the three models with the full training data

The matrices have the same shape for all three models. Almost every error is between neighbouring tiers: of the 924 errors of TabPFN, 922 were Low/Moderate or Moderate/High confusions, and only 2 were Low records predicted as High and 0 were High records predicted as Low. Table 4.4 gives the same counts for all nine runs. The extreme confusions were most frequent for CatBoost with 500 rows and almost disappeared with 2,000 rows or more.

Table 4.4: Errors of each run and their distance from the true tier

Training rows	Model	Errors	Between neighbouring tiers	Neighbouring (%)	Low predicted High	High predicted Low
500	CatBoost	1324	1286	97.1	15	23
500	TabPFN	999	994	99.5	3	2
500	TabFM	1001	995	99.4	3	3
2,000	CatBoost	1082	1074	99.3	2	6
2,000	TabPFN	939	937	99.8	2	0
2,000	TabFM	932	930	99.8	2	0
17,284	CatBoost	942	939	99.7	2	1
17,284	TabPFN	924	922	99.8	2	0
17,284	TabFM	922	920	99.8	2	0

The ordinal-aware measures in Table 4.5 lead to the same conclusion. Cohen's kappa was about 0.66 for the three models with the full data, which indicates substantial agreement, and the quadratic weighted kappa was about 0.81 because the few large errors are heavily penalised. In every run at least 99.1% of the predictions were within one tier of the true tier, and the mean absolute error was below 0.25 tiers with the full data.

Table 4.5: Ordinal-aware measures for the nine runs

Training rows	Model	Cohen's kappa	Quadratic weighted kappa	Within one tier (%)	MAE (tiers)
500	CatBoost	0.5078	0.7017	99.12	0.3151
500	TabPFN	0.6347	0.7953	99.88	0.2323
500	TabFM	0.6341	0.7949	99.86	0.2330
2,000	CatBoost	0.6007	0.7713	99.81	0.2522
2,000	TabPFN	0.6567	0.8073	99.95	0.2177
2,000	TabFM	0.6594	0.8083	99.95	0.2161
17,284	CatBoost	0.6555	0.8074	99.93	0.2186
17,284	TabPFN	0.6620	0.8108	99.95	0.2143
17,284	TabFM	0.6628	0.8109	99.95	0.2138

4.1.3 Statistical significance

McNemar's test compares two models on the records where exactly one of them is correct. Table 4.6 gives the counts b (only the first model correct) and c (only the second model correct), the test statistic and the p-value for the three pairs of models at each training size.

Table 4.6: Pairwise McNemar's test at each training size (4,322 test records)

Training rows	Comparison	b: only first correct	c: only second correct	χ^2	p-value	Significant at 0.05
500	CatBoost vs TabPFN	272	597	120.80	< 0.0001	Yes
500	CatBoost vs TabFM	280	603	117.42	< 0.0001	Yes
500	TabPFN vs TabFM	71	69	0.01	0.9326	No
2,000	CatBoost vs TabPFN	194	337	37.97	< 0.0001	Yes
2,000	CatBoost vs TabFM	196	346	40.96	< 0.0001	Yes
2,000	TabPFN vs TabFM	60	67	0.28	0.5944	No
17,284	CatBoost vs TabPFN	111	129	1.20	0.2725	No
17,284	CatBoost vs TabFM	114	134	1.46	0.2276	No
17,284	TabPFN vs TabFM	33	35	0.01	0.9035	No

With 500 and with 2,000 training rows, both foundation models were significantly better than CatBoost ($p < 0.0001$ in all four comparisons). For 500 rows, TabPFN was right and CatBoost wrong on 597 records, and the reverse held on only 272. With the full training data the differences disappeared: CatBoost against TabPFN gave $p = 0.27$ and CatBoost against TabFM gave $p = 0.23$, so no advantage of either foundation model over the tuned baseline could be detected. TabPFN and TabFM did not differ significantly at any size ($p = 0.93, 0.59$ and 0.90). Nine tests were run, and a Bonferroni-corrected level of $0.05/9 = 0.0056$ leaves the four significant results (all with $p < 0.0001$) unchanged. A non-significant result does not prove that the models are equivalent, only that no difference was detected with 4,322 test records.

Table 4.7: Accuracy (%) with 95% Wilson score confidence intervals

Training rows	Model	Accuracy	95% confidence interval
500	CatBoost	69.37	67.98 – 70.72
500	TabPFN	76.89	75.61 – 78.12
500	TabFM	76.84	75.56 – 78.07
2,000	CatBoost	74.97	73.65 – 76.23
2,000	TabPFN	78.27	77.02 – 79.48
2,000	TabFM	78.44	77.18 – 79.64
17,284	CatBoost	78.20	76.95 – 79.41
17,284	TabPFN	78.62	77.37 – 79.82
17,284	TabFM	78.67	77.42 – 79.86

Table 4.7 shows the uncertainty that comes from the finite test sample. Each interval is about 2.5 points wide. The three intervals with the full data overlap almost completely, whereas at 500 rows the interval of CatBoost lies far below those of the foundation models. The intervals do not include the variation from other data splits or seeds, which was not measured.

4.1.4 Quality of the predicted probabilities

So far the models were compared by the tier they predict. Each model also returns a probability for every tier, and the quality of these probabilities matters when a risk estimate is shown to a person. Table 4.8 gives the log loss (the mean negative logarithm of the probability given to the true tier, lower is better), the Brier score (the mean squared distance between the probability vector and the true tier, lower is better) and the mean confidence (the average of the highest probability), which can be compared with the accuracy.

Table 4.8: Log loss, Brier score and mean confidence of the nine runs

Training rows	Model	Log loss	Brier score	Mean confidence	Accuracy
500	CatBoost	0.6616	0.3996	0.6521	0.6937
500	TabPFN	0.5025	0.3099	0.7770	0.7689
500	TabFM	0.5072	0.3123	0.7969	0.7684
2,000	CatBoost	0.5589	0.3395	0.6922	0.7497
2,000	TabPFN	0.4807	0.2978	0.7924	0.7827
2,000	TabFM	0.4911	0.3019	0.8104	0.7844
17,284	CatBoost	0.4898	0.3028	0.7639	0.7820
17,284	TabPFN	0.4747	0.2947	0.7863	0.7862
17,284	TabFM	0.4785	0.2959	0.8054	0.7867

With the full data the two foundation models gave slightly better probabilities than CatBoost (log loss 0.4747 for TabPFN and 0.4785 for TabFM, against 0.4898 for CatBoost). The gap was large with 500 rows: the log loss of CatBoost was 0.6616 against 0.5025 for TabPFN, and its Brier score was 0.3996 against 0.3099. A model trained on 500 rows was also under-confident (mean confidence 0.652 for an accuracy of 0.694), whereas TabPFN with 500 rows had a mean confidence of 0.777 for an accuracy of 0.769, which is close. Better probabilities are an advantage of the foundation models when a numerical risk and not only a tier is reported, and this advantage, like the one in accuracy, shrinks with more data. No calibration method was applied to any model, and the probabilities were stored with four decimals, which is enough for these measures.

4.1.5 Overlap of the errors of the three models

The differences between the models are small compared with the number of errors that they share. Table 4.9 counts, for each training size, the test records that were classified correctly by all three models, by exactly two, by exactly one and by none.

Table 4.9: Number of test records classified correctly by all, two, one or none of the three models

Training rows	All three correct	Exactly two correct	Exactly one correct	None correct	None correct (%)
500	2,692	620	326	684	15.8
2,000	3,013	374	226	709	16.4
17,284	3,251	147	131	793	18.3

With 500 rows, 946 test records (21.9%) were classified correctly by some models and not by others, and with the full data only 278 records (6.4%). With the full data, 793 records (18.3%) were misclassified by all three models, which is 84% of the errors of CatBoost, 86% of those of TabPFN and 86% of those of TabFM. The models therefore make mostly the same mistakes, and the more data they have, the more alike their mistakes are. This points to a limit set by the data themselves, such as records close to a tier boundary or noise in the data, and not by the choice of model, which is the reason why three very different methods end at almost the same accuracy.

4.1.6 Performance in subgroups

The accuracy of the models was also checked in subgroups of the test partition (Table 4.10). The gender "Other" could not be examined because none of its 12 records fell in the test partition, and age was grouped into three bands.

Table 4.10: Accuracy (%) in subgroups of the test partition

Training rows	Model	Female (n = 1,941)	Male (n = 2,381)	Age 20–29 (n = 1,978)	Age 30–39 (n = 1,269)	Age 40–55 (n = 1,075)
500	CatBoost	69.81	69.00	71.49	67.77	67.35
500	TabPFN	77.49	76.40	77.40	76.04	76.93
500	TabFM	77.74	76.10	77.45	76.52	76.09
17,284	CatBoost	78.83	77.70	78.92	78.17	76.93
17,284	TabPFN	79.03	78.29	79.32	78.17	77.86
17,284	TabFM	78.88	78.50	79.47	78.33	77.58

With the full data the accuracy differed by at most 1.1 points between women and men and by at most 2.0 points between the age bands. Accuracy was a little higher for women than for men in every run, and it fell slightly from the youngest to the oldest band in all three models with the full data. These differences are small and were not tested, because the

subgroups are small and the dataset came from a single company, so they should be read as a check that no group was served much worse and not as evidence about fairness in real patients.

4.2 Effect of training context size

Table 4.11 and Fig. 4.2 show how the performance changed with the number of training rows. CatBoost gained a great deal from more data, whereas the two foundation models gained little.

Table 4.11: Accuracy (%) and macro-F1 by training size

Model	Acc. 500	Acc. 2,000	Acc. Full	Change (points)	F1 500	F1 2,000	F1 Full	Change
CatBoost	69.37	74.97	78.20	+8.83	0.6648	0.7377	0.7744	+0.1096
TabPFN	76.89	78.27	78.62	+1.73	0.7616	0.7757	0.7785	+0.0169
TabFM	76.84	78.44	78.67	+1.83	0.7603	0.7779	0.7802	+0.0199



Fig. 4.2: Accuracy, macro-F1 and ROC-AUC of the three models against the number of training rows (logarithmic axis)

From 500 rows to the full training data, the accuracy of CatBoost rose by 8.83 percentage points, whereas TabPFN rose by 1.73 and TabFM by 1.83 points. CatBoost therefore depended about 5 times more on the amount of data. The steps were also uneven: from 500 to 2,000 rows CatBoost gained 5.60 points and from 2,000 rows to the full data a further 3.23 points, while the foundation models gained about 1.5 and 0.3 points.

Two comparisons show how efficiently each model used its data. First, the foundation models with 500 rows (76.89% and 76.84%) were more accurate than CatBoost with 2,000 rows (74.97%), so they reached a given accuracy with a quarter of the rows or less. Second, CatBoost needed all 17,284 rows to reach the accuracy (78.20%) that TabPFN (78.27%) and TabFM (78.44%) had with 2,000 rows, which is 8.6 times more data. Because two independently developed foundation models behaved in the same way, the behaviour does not seem to be specific to one design, although only one dataset was tested.

The accuracy of the foundation models flattened at about 78.6% to 78.7%, so more context gave little further gain on this dataset.

The ranking of TabPFN and TabFM with each other was not reliable at any size: TabFM was slightly higher at 2,000 and full rows and TabPFN slightly higher at 500 rows, and none of the differences was significant (Section 4.1.3).

The per-tier F1-scores in Table 4.12 show where the gain from more data came from.

Table 4.12: F1-score by risk tier and training size

Model	Tier	500 rows	2,000 rows	17,284 rows	Change 500 to Full
CatBoost	Low	0.8123	0.8331	0.8537	+0.0415
	Moderate	0.5811	0.6580	0.7007	+0.1196
	High	0.6011	0.7220	0.7689	+0.1678
TabPFN	Low	0.8428	0.8528	0.8568	+0.0140
	Moderate	0.6814	0.7054	0.7080	+0.0266
	High	0.7608	0.7689	0.7708	+0.0101
TabFM	Low	0.8444	0.8526	0.8545	+0.0101
	Moderate	0.6800	0.7090	0.7100	+0.0300
	High	0.7566	0.7723	0.7762	+0.0196

For CatBoost the largest gain was in the High tier (F1 from 0.601 to 0.769), whereas TabPFN and TabFM gained most in the Moderate and Moderate tiers (+0.027 and +0.030). With 500 rows the recall of the High tier was only 0.52 for CatBoost against 0.73 for TabPFN and 0.73 for TabFM, and the recall of the Moderate tier was 0.59 against 0.70 and 0.70. A tuned model with few rows therefore missed many patients of the middle and upper tiers, which is the clinically important failure, and the foundation models missed far fewer.

4.3 Explainability of the predictions

4.3.1 Relationship between the features and the risk tier in the data

Before the model explanations are read, Table 4.13 and Table 4.14 show what the data themselves say. The continuous features are ordered by the absolute value of the Spearman correlation with the risk tier, and the mean of each feature in each tier is given.

Table 4.13: Mean of the continuous predictors by risk tier and Spearman correlation with the tier

Predictor	Low	Moderate	High	Spearman ρ
stress_level	15.40	19.05	22.18	+0.33
iron	108.34	94.82	82.64	-0.31
total_protein	7.39	7.13	6.91	-0.30
vitamin_d	31.23	26.92	23.18	-0.26
alt_liver	22.20	26.64	31.73	+0.25
calcium	9.44	9.24	9.08	-0.23
manganese	8.56	7.85	7.09	-0.18
body_water_content	56.05	54.62	53.22	-0.14
age	31.73	33.06	34.90	+0.12
total_keratine	49.91	50.10	49.40	-0.01
hair_texture	50.02	50.46	49.97	+0.00

Table 4.14: Share of records (%) with each binary flag, by risk tier

Flag	Low	Moderate	High	Spearman ρ
stress	34.2	42.8	49.9	+0.12
chemical_use	33.9	42.6	48.6	+0.12
family_hair_fall_history	28.9	37.1	42.5	+0.11
anemia	19.8	26.3	32.5	+0.11
chronic_illness	25.1	31.7	37.5	+0.10
sleep_disturbance	31.6	36.2	41.5	+0.08
late_night_sleep	36.5	41.3	46.4	+0.08
water_reason	27.0	30.6	34.6	+0.06

The strongest single relationships were with stress_level ($\rho = +0.33$, higher in higher tiers), iron ($\rho = -0.31$), total_protein ($\rho = -0.30$), vitamin_d ($\rho = -0.26$), alt_liver ($\rho = +0.25$) and calcium ($\rho = -0.23$). Iron, protein, vitamin D and calcium fall from the Low to the High tier, and stress and the liver enzyme ALT rise. Every binary flag was more common in higher tiers. These correlations are moderate, so no single feature separates the tiers, which agrees with the argument in Section 1.1.

4.3.2 Global feature importance

Fig. 4.3 to Fig. 4.5 show the global importance of the features of the three models with the full training data, measured as the mean absolute SHAP value averaged over the three classes. The values are on different scales, because CatBoost was explained with exact tree values on 1,000 records and the foundation models with kernel estimates on 30 and 20 records, so the models are compared by rank and not by the size of the bars.

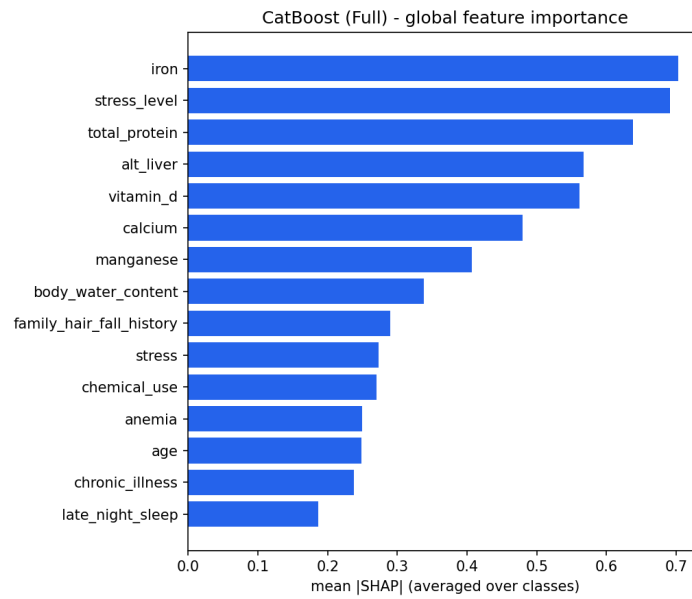


Fig. 4.3: CatBoost global feature importance (SHAP), full training data

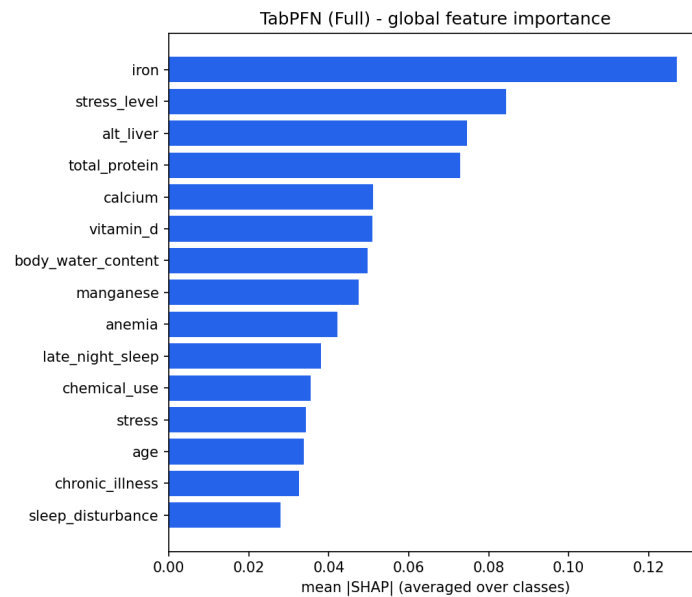


Fig. 4.4: TabPFN global feature importance (SHAP), full training data

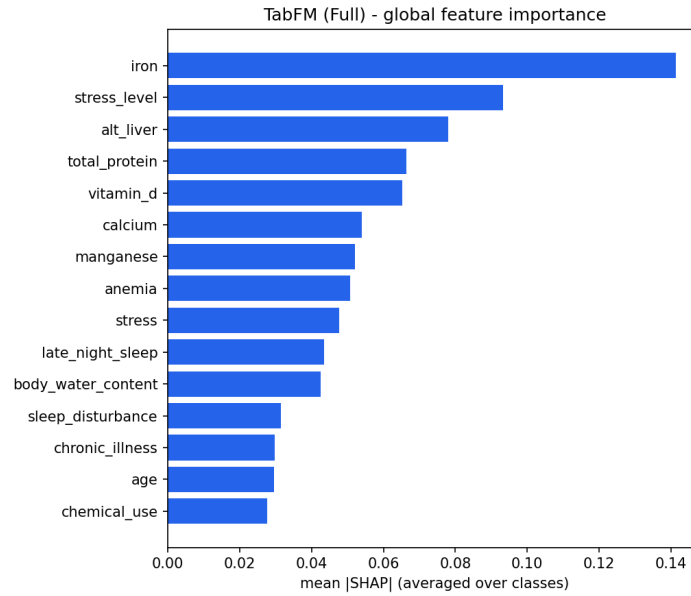


Fig. 4.5: TabFM global feature importance (SHAP), full training data

Table 4.15: Ten most important features with the full training data, by mean absolute SHAP value

Rank	CatBoost	TabPFN	TabFM
1	iron	iron	iron
2	stress_level	stress_level	stress_level
3	total_protein	alt_liver	alt_liver
4	alt_liver	total_protein	total_protein
5	vitamin_d	calcium	vitamin_d
6	calcium	vitamin_d	calcium
7	manganese	body_water_content	manganese
8	body_water_content	manganese	anemia
9	family_hair_fall_history	anemia	stress
10	stress	late_night_sleep	late_night_sleep

The same six features occupied the first six places in all three models with the full data, in a slightly different order: alt_liver, calcium, iron, stress_level, total_protein, vitamin_d. The seventh place was manganese for CatBoost and TabFM and body_water_content for TabPFN, in which manganese was eighth. Iron and stress_level were the two most important features in every model, and the demographic, hair-condition and binary features contributed less individually. Table 4.16 shows that this holds at the smaller training sizes as well: iron and stress_level are the first two features in all nine analyses.

Table 4.16: Five most important features in each of the nine SHAP analyses

Model	Training rows	Top five features (in order)
CatBoost	500	stress_level, iron, calcium, total_protein, vitamin_d
CatBoost	2,000	stress_level, iron, total_protein, alt_liver, vitamin_d
CatBoost	17,284	iron, stress_level, total_protein, alt_liver, vitamin_d
TabPFN	500	iron, stress_level, calcium, total_protein, vitamin_d
TabPFN	2,000	iron, stress_level, total_protein, alt_liver, calcium
TabPFN	17,284	iron, stress_level, alt_liver, total_protein, calcium
TabFM	500	iron, stress_level, calcium, total_protein, alt_liver
TabFM	2,000	iron, stress_level, alt_liver, total_protein, stress
TabFM	17,284	iron, stress_level, alt_liver, total_protein, vitamin_d

The set of the six leading features was the same in 8 of the nine analyses. The exception was TabFM with 2,000 rows, in which the yes/no feature stress took the fifth place and vitamin_d moved down to seventh. The two features that always led were iron and stress_level; in CatBoost with 500 and 2,000 rows stress_level was first and iron second, and in the other seven analyses the order was reversed.

4.3.3 Direction of the effects

The beeswarm plots in Fig. 4.6 to Fig. 4.8 show the direction of each effect for the High risk class. Each dot is one explained record, its position on the horizontal axis is the contribution of the feature to the High-risk output (right means towards High risk), and its colour is the value of the feature (red is high, blue is low).

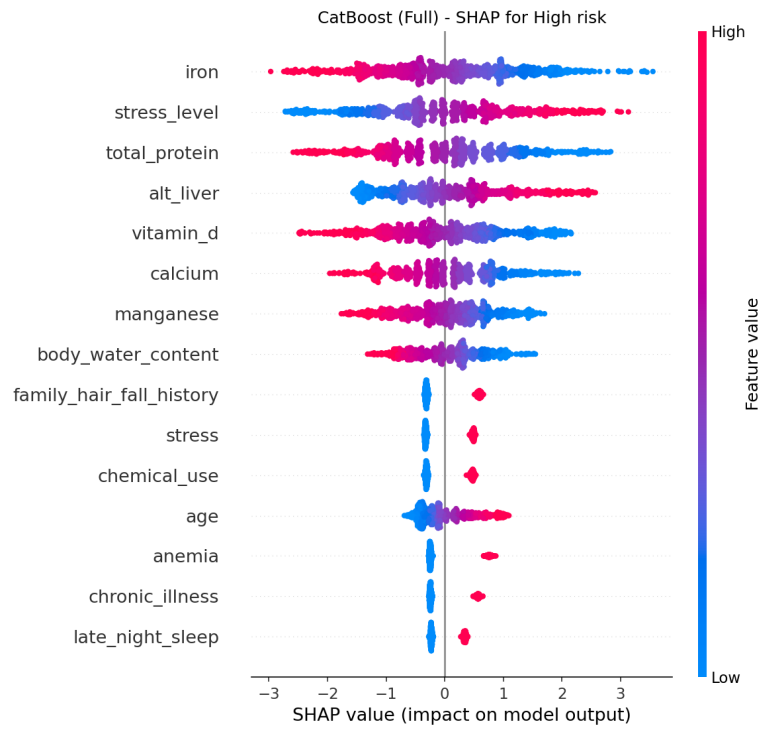


Fig. 4.6: CatBoost SHAP beeswarm plot for the High risk class, full training data

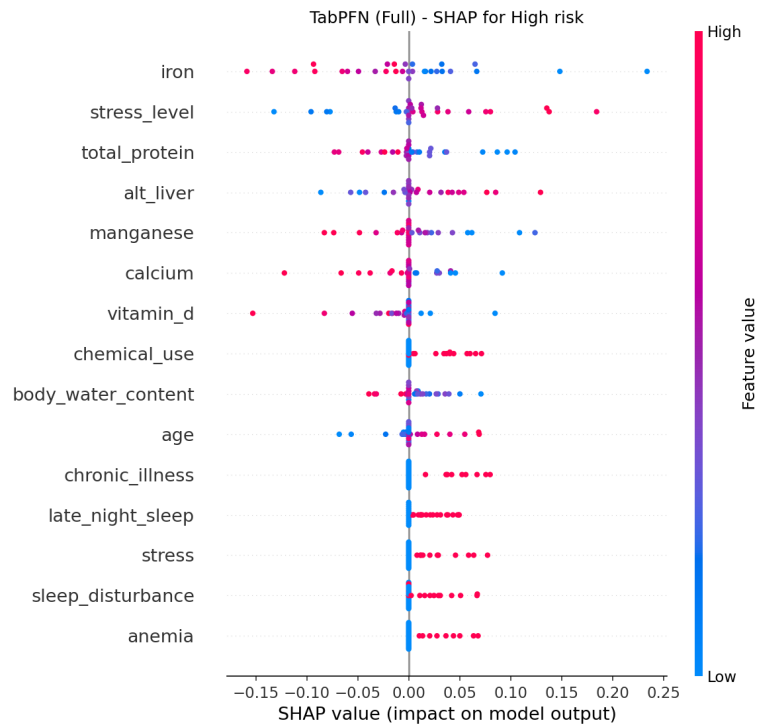


Fig. 4.7: TabPFN SHAP beeswarm plot for the High risk class, full training data

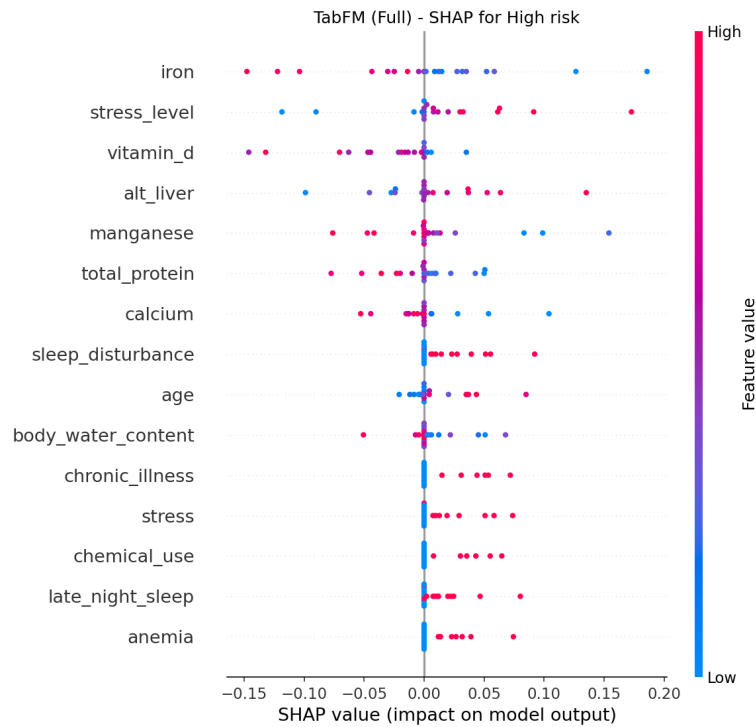


Fig. 4.8: TabFM SHAP beeswarm plot for the High risk class, full training data

The direction was the same in all three models. Low iron values (blue dots on the right) and high stress_level values (red dots on the right) pushed the prediction towards High risk. Higher total_protein, vitamin_d, calcium and manganese values (red dots on the left) pushed it away from High risk, and higher alt_liver values pushed it towards High risk. The yes/no features (chronic_illness, anemia, stress, late_night_sleep, sleep_disturbance, chemical_use and family_hair_fall_history) formed two clusters, with the value 1 always on the side of higher risk. Older age also pushed towards High risk. These directions match the signs of the correlations in Table 4.13 and Table 4.14.

4.3.4 Differences between the rankings

Table 4.17: Spearman rank correlation of the SHAP feature rankings of the three models (full training data)

	CatBoost	TabPFN	TabFM
CatBoost	1.000	0.926	0.896
TabPFN	0.926	1.000	0.956
TabFM	0.896	0.956	1.000

The rankings of all 20 features agreed closely between the models (Spearman correlation 0.90 to 0.96, Table 4.17). The differences are in the middle of the ranking, where the importance values are close together and, for the foundation models, are based on only 20 or 30 records. For example, the feature stress appeared among the five leading features of

TabFM with 2,000 rows only, and manganese and body_water_content changed places between models. Such changes should not be interpreted. Only the shared set of leading features and the direction of their effects are reliable, which is why the conclusions in Chapter 5 rest on them.

This finding must be read together with the data. The dataset is a single company dataset (Section 3.7). Agreement between the models and with the correlations in Section 4.3.1 shows that all three models recovered the relationships built into the dataset. It is not new clinical evidence about hair loss, and it cannot be assumed to hold in real patients.

4.4 Computational cost and implementation limitations

4.4.1 Run time

Table 4.18 gives the time to fit or set up each model and to predict the 4,322 test records, as recorded in the runs on the Kaggle GPU (CatBoost on the CPU). Fig. 4.9 shows the sum of the two on a logarithmic axis.

Table 4.18: Fit and prediction time of the nine runs (seconds)

Training rows	Model	Fit or context set-up (s)	Prediction of 4,322 records (s)	Per record (ms)
500	CatBoost	0.8	0.0	0.0
500	TabPFN	18.6	6.7	1.6
500	TabFM	15.1	not measured*	-
2,000	CatBoost	1.8	0.0	0.0
2,000	TabPFN	2.3	17.9	4.1
2,000	TabFM	15.9	209.3	48.4
17,284	CatBoost	3.3	0.0	0.0
17,284	TabPFN	2.5	289.5	67.0
17,284	TabFM	15.1	3,082.2	713.1

* The TabFM run with 500 rows was resumed from a saved checkpoint, so the prediction time was not measured. Fit time for the foundation models includes loading the model weights.

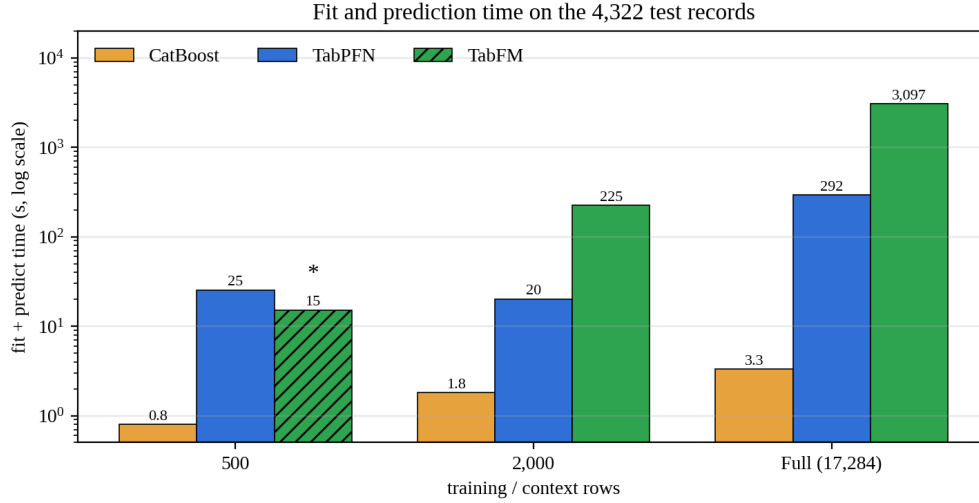


Fig. 4.9: Fit and prediction time of the three models at each training size (logarithmic axis). The hatched bar has no prediction time (resumed run).

CatBoost was by far the cheapest: fitting took 0.8 to 3.3 seconds and predicting the test records took less than a second. The whole CatBoost step, including the grid search with cross-validation, took between 158 and 358 seconds. The foundation models have almost no fitting cost, because they only store the rows, but their prediction cost grows with the size of the context. TabPFN needed 6.7 s, 17.9 s and 289.5 s for 500, 2,000 and 17,284 rows, an increase of 43 times for 34.6 times more rows. TabFM needed 209.3 s with 2,000 rows and 3,082.2 s (about 51 minutes) with the full data, which is 10.6 times the time of TabPFN, and about 713 ms per record against 67 ms for TabPFN. Predicting the test records with TabFM took roughly 934 times as long as fitting CatBoost on the same data.

Explaining the models was much more expensive than predicting with them (Table 4.19). TreeExplainer took a fraction of a second for 1,000 records of CatBoost. Kernel SHAP needed 100 samples for each explained record, and with the full context each explained record took 108 seconds for TabPFN and 425 seconds for TabFM. This is why only 30 and 20 records could be explained for the foundation models.

Table 4.19: Time of the SHAP analyses

Model	Training rows	Explainer	Records explained	Time (s)	Time per record (s)
CatBoost	500	TreeExplainer	1000	0.3	0.00
CatBoost	2,000	TreeExplainer	1000	0.3	0.00
CatBoost	17,284	TreeExplainer	1000	0.3	0.00
TabPFN	500	KernelExplainer	30	65.8	2.19
TabPFN	2,000	KernelExplainer	30	193.0	6.43
TabPFN	17,284	KernelExplainer	30	3,251.3	108.38
TabFM	500	KernelExplainer	20	158.0	7.90
TabFM	2,000	KernelExplainer	20	581.8	29.09
TabFM	17,284	KernelExplainer	20	8,491.4	424.57

4.4.2 Software problems and workarounds

The runs exposed practical problems. They are recorded here because they affect anyone who plans to repeat the study, and because they are usually left out of benchmark reports.

- **Memory.** TabFM with the full 17,284-row context could not be run on the local computer with 16 GB of RAM and no GPU: the kernel ran out of memory and stopped, and the first 200 records took 3,240 seconds (16 seconds per record), which projects to about 19 hours for the test partition. It was therefore run on the Kaggle GPU. Even there, the default attention implementation exceeded the memory of the T4, so the attention was evaluated in chunks of 1,024 queries, the column chunk size was set to 4, and the prediction batch size was halved automatically after any out-of-memory error. These settings change the memory use only.
- **Dependencies.** The TabFM package required a different array library that conflicted with the notebook image, so that library was removed and the PyTorch backend was used.
- **Single-row input.** TabPFN and TabFM failed on a single query row. The SHAP wrapper pads a one-row query with a copy and keeps the first prediction.
- **Long runs and interruptions.** Predictions were saved in batches, so that runs of many hours could resume after an interruption. A resumed run has no valid prediction time, which is the case for TabFM with 500 rows.
- **Licence and downloads.** TabPFN needs a licence token, and the TabFM weights are large (about 6.5 GB) and have to be downloaded once.

- **Bookkeeping.** In the Lab, a run that was stopped by hand was at one point stored as successful. A check was added so that a stopped run is stored as an error, and the affected result was restored from the saved files.

These problems come mainly from the recency and the size of the foundation models and not from the modelling approach. CatBoost ran without any of them.

4.5 Discussion

4.5.1 Plausibility of the findings

A ceiling of about 78% to 79% accuracy for all three models, whatever their design, suggests that the limit is set by the data and not by the models. The Moderate tier, which lies between the other two, was the hardest for every model, and almost all errors were between neighbouring tiers. This is the pattern expected when a continuous risk is cut into three ordered tiers: records close to a boundary are easily placed on the wrong side, whereas confusing Low with High is almost never seen. The agreement of the three models in their leading features and in the direction of their effects, and its agreement with the correlations in the data, shows that the models learned the same relationships. Because the dataset came from a single company, this agreement shows what the models can recover from such data and is not clinical evidence.

4.5.2 Comparison with similar studies

The hair loss studies reviewed in Section 2.1 used different data, targets and metrics, so their accuracies cannot be compared with the values here. What can be compared is the behaviour of the models. The finding that a foundation model beats a tuned tree ensemble on small data agrees with the report of Hollmann et al. [20] for datasets of up to 10,000 samples, and the finding that the gap closes at 17,284 rows is consistent with that report being limited to smaller tables. The zero-shot competitiveness of TabFM reported by its authors [9] and by the independent evaluation [10] is confirmed here at every size, but no advantage over TabPFN or over a tuned CatBoost with the full data was found. The result of Patel et al. [4], that Random Forest outperformed CatBoost, cannot be compared with this study, which did not evaluate Random Forest. The conclusion of Sai et al. [3] that an ensemble method beat individual classifiers is in line with the strength of the boosted ensemble at large sizes. The tree-versus-deep-learning results of [17] and [18] are matched

by the strength of CatBoost with all the data, and the advantage of pretrained models with little data adds a case where the balance favours the neural approach.

4.5.3 Implications and cautions

Four implications follow, each with a caution. First, where labelled data are limited (a few thousand rows or fewer), a zero-shot foundation model such as TabPFN is a reasonable first choice, since it reached the accuracy of a tuned CatBoost without tuning and lost far less accuracy when the data were reduced. The caution is that this advantage was measured on one company dataset. Second, with plenty of data the tuned gradient-boosted model is as accurate and thousands of times cheaper to run, so the extra cost of a foundation model is hard to justify. Third, the cost of the foundation models grows with the context and their explanation with Kernel SHAP is very expensive, which limits where they can be used. Fourth, explanations by SHAP could be produced for all three models and agreed with each other, but they describe the models and not causes.

A practical point that is not visible in the accuracy is deployment. A fitted CatBoost model is a small file that predicts in milliseconds without the training data and without a GPU. TabPFN and TabFM need the training rows at prediction time, TabFM needs a large weight file (about 6.5 GB), and both needed a GPU to predict the test records in a reasonable time with the full context. For a tool that gives a risk to one person at a time, as in a clinic, this favours CatBoost unless the training data are very few.

4.5.4 Relation of the results to the research questions

- **Research Question 1.** With the full data no significant difference between CatBoost, TabPFN and TabFM was found (p between 0.23 and 0.90). With 500 and 2,000 rows both foundation models were significantly more accurate than CatBoost.
- **Research Question 2.** CatBoost gained 8.8 accuracy points from 500 rows to the full data, whereas TabPFN and TabFM gained 1.7 and 1.8. The foundation models used limited data far more efficiently.
- **Research Question 3.** Iron and stress_level were the two leading features in every analysis, with total_protein, alt_liver, vitamin_d and calcium completing the first six places in eight of the nine analyses, and with the same direction of effect in all three models and in agreement with the correlations in the data.

- **Research Question 4.** CatBoost took seconds. TabPFN took about 5 minutes and TabFM about 51 minutes to predict the test records with the full context on a T4 GPU, and TabFM could not be run on a 16 GB CPU computer. Kernel SHAP for the foundation models took up to 2.4 hours for 20 records.

4.5.5 Threats to the validity of the findings

Four kinds of threat limit how far the findings can be trusted, and they are listed here so that the conclusions of Chapter 5 are read with them in mind.

Construct validity. The target is a three-tier label in a company dataset and not a clinical diagnosis, so accuracy measures agreement with the labels built into the data and not with true hair loss. The tiers are cut from a continuous risk, so labels near a boundary are partly arbitrary, which the large share of errors made by all three models (Section 4.1.5) illustrates.

Internal validity. The models were compared on the same records and the same split, the test partition was not used for tuning or stopping, and the baseline was tuned. The CatBoost grid was small, the foundation models were used with default settings, the cloud runs did not fix the versions of TabPFN and SHAP, and TabFM was run in a memory-saving configuration. Reruns of the same step differed in the third decimal place (Section 3.10), so differences of a few tenths of a point between models cannot be interpreted, and all differences with the full training data are of that size.

External validity. There was one dataset, from a single company, with adults aged 20 to 55 and 20 mostly numeric predictors. The size of the advantage of the foundation models, the point at which CatBoost catches up and the rank of the features may all depend on the number and type of the features, on the class balance and on how the labels were built. The results cannot be assumed for real patients or for other tabular problems.

Conclusion validity. There was one split and one seed. McNemar's test on 4,322 records has limited power for small differences: between CatBoost and TabPFN with the full data only 240 records were classified correctly by one model and not by the other, so a difference of about 31 records (0.7 accuracy points) would have been needed to reach significance. A non-significant result at full size therefore means that no difference larger than about this size was found, and it does not show that the models are equal. Nine tests were run without correction in the main analysis, although the conclusions do not change

under a Bonferroni correction (Section 4.1.3), and the Wilson intervals show an uncertainty of about 1.3 points in each accuracy value.

CHAPTER 5

CONCLUSION AND RECOMMENDATIONS

5.1 Conclusion

This study set out to benchmark CatBoost, TabPFN and TabFM for explainable, multi-tier (Low, Moderate, High) hair fall risk stratification from structured clinical and lifestyle data. It was motivated by three gaps identified in Section 1.2: the evidence on the best model for hair fall data was inconsistent, tabular foundation models had not been tested on this problem, and complex models were hard to interpret. The four research questions of Section 1.3 are answered below, using the results of Chapter 4.

Predictive performance (Research Question 1). With all 17,284 training rows the three models reached close results on the 4,322-record test partition. Accuracy was 78.20% for CatBoost, 78.62% for TabPFN and 78.67% for TabFM, and macro-F1 was 0.7744, 0.7785 and 0.7802. McNemar's test found no significant difference between any pair of models at the 0.05 level (p between 0.23 and 0.90). On this dataset, therefore, no advantage of the foundation models over a tuned gradient-boosted model was detected when all the data were used. This does not prove that the models are equivalent. With 500 and with 2,000 training rows, however, both foundation models were significantly more accurate than CatBoost ($p < 0.0001$). Almost every error, for every model, was between neighbouring tiers, and the Moderate tier was the hardest to classify.

Effect of training size (Research Question 2). From 500 rows to the full training data, the accuracy of CatBoost rose by 8.83 points, whereas that of TabPFN rose by 1.73 and that of TabFM by 1.83 points. CatBoost needed all 17,284 rows to reach the accuracy that the foundation models had with 2,000 rows, and the foundation models with 500 rows were more accurate than CatBoost with 2,000 rows. The tuned gradient-boosted model therefore depended most on the amount of training data. Because two independently developed foundation models behaved in the same way, the behaviour does not seem to be specific to one design, although only one dataset was tested. The ranking of TabFM and TabPFN with each other was not reliable at any size.

Explainability (Research Question 3). SHAP explanations were produced for all three models at all three training sizes. Iron and stress_level were the two most influential features in every one of the nine analyses, and total_protein, alt_liver, vitamin_d and

calcium completed the first six places in eight of them. The direction of the effects was the same in all three models: higher stress_level and alt_liver values and the presence of the yes/no risk factors raised the predicted High risk contribution, and higher iron, total_protein, vitamin_d, calcium and manganese values lowered it. This agrees with the correlations measured in the data and with the relationships used to build the dataset, so it shows that the models recovered those relationships. It is not new clinical evidence.

Computational cost and implementation (Research Question 4). CatBoost was fitted in a few seconds and its predictions were immediate. With the full context on a T4 GPU, TabPFN needed about 5 minutes and TabFM about 51 minutes to predict the test records, and TabFM could not be run at all on a computer with 16 GB of RAM and no GPU. Explaining the foundation models with Kernel SHAP took up to 54 minutes for 30 records (TabPFN) and 2.4 hours for 20 records (TabFM). The runs also exposed memory limits, package conflicts and single-row failures that needed workarounds. These problems came from the size and recency of the software and not from the modelling approaches.

In summary, on this dataset a tuned gradient-boosted model and two zero-shot foundation models gave comparable accuracy when all the data were available, the foundation models were much more efficient when data were limited, and all three could be explained with SHAP, but the foundation models cost far more to run and to explain. This meets the general objective of the study, within the limits described in Section 5.2.

The problem stated in Section 1.2 was that the evidence on the best model for hair fall data was inconsistent, that tabular foundation models had not been tested on it, and that complex models were hard to interpret. The general objective of Section 1.4 was to benchmark the three models for explainable multi-tier hair fall risk stratification, and each of the four specific objectives was met by the analyses in Sections 4.1 to 4.4. The comparison was made on one dataset, so the conclusions describe the behaviour of the models on this dataset and should not be generalised beyond it.

The contribution of the study can be stated in four points. It is a benchmark of two independently developed tabular foundation models against a tuned gradient-boosted model on multi-tier hair fall risk classification, a comparison that the reviewed literature did not contain (Section 2.8). It compares the three models at three matched training sizes on the complete test partition, and shows how the comparison changes from limited to full data. It applies one explanation method to three different architectures at every size and checks the explanations against the relationships measured in the data (Section 4.3). It also

documents the run time and the software problems met with a recently released model, which may help others who plan to use it.

5.2 Limitations of the study

The conclusions must be read with the following limitations.

1. The dataset came from a single company (Section 3.7). It was not drawn from a clinical registry, so the results show how the models perform on this dataset and are not the validation of a diagnostic tool.
2. All results come from one train/test split with one random seed. Confidence intervals were given only for accuracy (Table 4.7) and describe the uncertainty from the finite test sample, not the variation between splits or seeds. Reruns of the same step differed in the third decimal place (Section 3.10).
3. The SHAP analyses of TabPFN and TabFM used only 30 and 20 test records and small background samples, so their attributions are less certain than CatBoost's exact values on 1,000 records. Only the leading features and the direction of their effects were interpreted. Patient-level (waterfall) explanations were not produced.
4. CatBoost ran on a CPU and the foundation models on a GPU, so the run times are not comparable across model families in absolute terms, and they come from shared cloud hardware. The prediction time of TabFM with 500 rows was not measured because the run was resumed from a checkpoint.
5. The versions of TabPFN and SHAP were not fixed in the cloud runs, and TabFM was evaluated in the memory-saving configuration described in Section 3.8.4.
6. The CatBoost grid was small (nine combinations), and TabPFN and TabFM were used with default settings, so a more thorough tuning of any of the three models could change the ranking.
7. The Moderate tier was the hardest to classify in all three models. The study offered an explanation based on the ordered structure of the target but did not test an ordinal model.

5.3 Recommendations

Recommendations for practice. These follow from the findings and apply to structured data of a similar kind.

1. Where labelled data are limited (a few thousand rows or fewer), a zero-shot tabular foundation model such as TabPFN is a reasonable first choice, since it reached or exceeded the accuracy of the tuned CatBoost without tuning and lost much less accuracy when the training data were reduced.
2. Where a large training set is available, a tuned gradient-boosted model such as CatBoost is a sound choice, since with all the data it could not be told apart from the foundation models and it costs seconds instead of minutes or hours.
3. The running time and memory of TabFM and TabPFN should be measured at the intended context size and on the intended hardware before an evaluation is planned, because their cost grew steeply with the context.
4. Feature-level explanations such as SHAP should accompany any risk prediction, and in this study they were available for all three models. Any use in a health setting should be as decision support and only after validation on real clinical data.

Recommendations for further research. These address the limitations above.

1. Repeat the experiment with repeated cross-validation or several random splits and seeds, and report confidence intervals for the differences between the models, to find out how reliable the small differences at full size are.
2. Compute SHAP values for the foundation models on more test records and with larger backgrounds, and add patient-level explanations, so that the feature order can be compared more closely.
3. Tune TabPFN and TabFM options and use a larger CatBoost search space, to test whether the ranking at full size changes.
4. Compare ordinal-aware models with the multi-class approach, to see whether they improve the classification of the Moderate tier.
5. Validate the framework on real clinical hair loss data when ethically cleared data become available.
6. Study ensembles of the three models, and distillation of the foundation models into lightweight models, since TabPFN and TabFM need their training data at prediction time.
7. Fix the versions of all packages and record the hardware of every run, so that the times can be compared exactly.

Each recommendation follows from a finding. The first two recommendations for practice rest on the results of Section 4.1 and Section 4.2, the third on the run times in Section 4.4, and the fourth on the explanations in Section 4.3. Each recommendation for further research addresses one of the limitations in Section 5.2. The recommendations are limited to the scope of this study and to structured data of a similar kind.

REFERENCES

- [1] A. Widowati, S. A. Yuniarto, and M. A. R. Syaipullah, "Explainable machine learning decision support system for multi-class hair loss severity classification," *International Journal of Technology in Education and Science*, vol. 10, no. 1, pp. 115–136, 2026.
- [2] M. Sayyad and S. Midhunchakkaravarthy, "Machine learning-based diagnostic model for Alopecia Areata prediction," *Journal of Dermatological Informatics*, vol. 15, no. 2, pp. 45–52, 2023.
- [3] C. N. V. Sai, E. Archana, B. Vivek, B. Dhanwanth, and V. K. S., "Enhancing Hairfall Prediction: A Comparative Analysis of Individual Algorithms and An Ensemble Method," *International Journal on Recent and Innovation Trends in Computing and Communication*, vol. 11, no. 6s, pp. 499–508, 2023.
- [4] S. Patel, V. Gupta, and N. Mehta, "Predictive Modeling of Hair Fall using Decision Tree and Random Forest Architectures," *EAI Endorsed Transactions on Pervasive Health and Technology*, vol. 11, pp. e1–e10, 2025.
- [5] K. Chen, L. Zhang, and R. Wang, "Predicting Hair Loss with AI: A Deep Learning Framework Combining Genetic and Scalp Health Data," in *Integrating AI, Machine Learning, and IoT in Bioinformatics: Innovations in Biotech and Medical Research*, CRC Press, pp. 100–107, 2024.
- [6] C. S. Shakeel, S. J. Khan, B. Chaudhry, S. F. Aijaz, and U. Hassan, "Classification Framework for Healthy Hairs and Alopecia Areata: A Machine Learning (ML) Approach," *Computational and Mathematical Methods in Medicine*, vol. 2021, Article ID 1102083, 2021.
- [7] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 31, 2018, pp. 6638–6648.
- [8] N. Hollmann, S. Müller, K. Eggenberger, and F. Hutter, "TabPFN: A transformer that solves small tabular classification problems in a second," in *Proc. 11th Int. Conf. Learning Representations (ICLR)*, 2023.
- [9] W. Kong and A. Das, "Introducing TabFM: A zero-shot foundation model for tabular data," *Google Research Blog*, Jun. 2026. [Online]. Available: <https://research.google/blog/introducing-tabfm-a-zero-shot-foundation-model-for-tabular-data/>. [Accessed: DD Mon. YYYY].
- [10] devYRPauli, "tabfm-evaluation: Independent 3-machine evaluation of Google TabFM," *GitHub repository*, Jul. 2026. [Online]. Available: <https://github.com/devYRPauli/tabfm-evaluation>. [Accessed: DD Mon. YYYY].
- [11] S. M. Lundberg and S.-I. Lee, "A unified approach to interpreting model predictions," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 4765–4774.

- [12] J. Anderson and M. Roberts, "Clinical feature analysis and survey-driven lifestyle metrics for predicting hair loss severity," *Applied Medical Informatics Review*, vol. 18, no. 1, pp. 78–89, 2025.
- [13] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [14] J. H. Friedman, "Greedy function approximation: A gradient boosting machine," *The Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [15] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [16] G. Ke et al., "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 3146–3154.
- [17] R. Shwartz-Ziv and A. Armon, "Tabular data: Deep learning is not all you need," *Information Fusion*, vol. 81, pp. 84–90, 2022.
- [18] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 507–520.
- [19] A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.
- [20] N. Hollmann et al., "Accurate predictions on small data with a tabular foundation model," *Nature*, vol. 637, pp. 319–326, 2025.
- [21] L. S. Shapley, "A value for n-person games," in *Contributions to the Theory of Games*, vol. II, H. W. Kuhn and A. W. Tucker, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1953, pp. 307–317.
- [22] S. M. Lundberg et al., "From local explanations to global understanding with explainable AI for trees," *Nature Machine Intelligence*, vol. 2, no. 1, pp. 56–67, 2020.
- [23] M. Sokolova and G. Lapalme, "A systematic analysis of performance measures for classification tasks," *Information Processing & Management*, vol. 45, no. 4, pp. 427–437, 2009.
- [24] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.
- [25] T. G. Dietterich, "Approximate statistical tests for comparing supervised classification learning algorithms," *Neural Computation*, vol. 10, no. 7, pp. 1895–1923, 1998.
- [26] E. Frank and M. Hall, "A simple approach to ordinal classification," in *Machine Learning: ECML 2001*, ser. Lecture Notes in Computer Science, vol. 2167. Berlin, Germany: Springer, 2001, pp. 145–156.

- [27] Company dataset (hair fall biomarker data), provided by the company for this study; not publicly available.
- [28] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
- [29] J. Cohen, "Weighted kappa: Nominal scale agreement with provision for scaled disagreement or partial credit," *Psychological Bulletin*, vol. 70, no. 4, pp. 213–220, 1968.
- [30] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.
- [31] F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

APPENDIX A

SOURCE CODE

This appendix gives the code of every step of the pipeline. Each step is a short Python script that is run in the Hairfall Lab (Section 3.8.9) after the shared module of Section A.1 is loaded. The scripts read the result files of the earlier steps from their step folders, and write their own results (metrics, predictions, figures, tables) to the folder of their step. The code is listed as it is stored in the project (folder backend/steps). A step is identified by its number: Steps 1 to 4 prepare the data, Steps 5 to 7 fit or run the three models at the three training sizes (a, b and c stand for 500, 2,000 and Full), Step 8 explains the models with SHAP, and Steps 9 and 10 compare the results.

A.1 Shared helper module (*lab.py*)

```
"""Shared helpers for the thesis steps.

The dataset is loaded into Postgres automatically when the backend starts, so steps
only ever *read* it. Every file a step writes to outputs/ is saved into
thesis_project/<Step>/ once the step finishes successfully.
"""
import json
import time
from pathlib import Path

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import psycpg
from sklearn.metrics import (ConfusionMatrixDisplay, accuracy_score, classification_report,
                             f1_score, roc_auc_score)
from sklearn.model_selection import train_test_split

DSN = "postgresql://lab:lab@127.0.0.1:5432/lab"
PROJECT = Path(__file__).resolve().parent.parent.parent / "thesis_project"
OUT = Path(__file__).resolve().parent / "outputs"
SEED = 42
CLASS_NAMES = ["Low", "Moderate", "High"]
SIZES = {"500": 500, "2000": 2000, "Full": None} # training-context sizes used in steps 5-8

def load_df(table: str) -> pd.DataFrame:
    """Read a table from the local Postgres (read-only use)."""
    with psycpg.connect(DSN) as conn:
        return pd.read_sql(f'SELECT * FROM "{table}"', conn)

def step_file(step: str, name: str) -> Path:
    """Path of a file saved by an earlier step, with a clear error if that step hasn't run."""
    p = PROJECT / step / name
    if not p.exists():
        raise FileNotFoundError(f"{name} not found - run {step} first.")
    return p

def test_rows():
    """Test rows every model is scored on: number in workspace/test_rows.txt ('all' = whole
    test set).
    Read fresh on each call, so editing the file needs no restart."""
    v = (OUT.parent / "test_rows.txt").read_text().strip().lower()
```

```

return None if v in ("all", "none", "") else int(v)

def load_split():
    """Train/test split from Step 4. The test set is the same stratified subset for every
    model."""
    tr = pd.read_csv(step_file("Step_04_TrainTestSplit", "train.csv"))
    te = pd.read_csv(step_file("Step_04_TrainTestSplit", "test.csv"))
    X_train, y_train = tr.drop(columns="hair_fall", tr["hair_fall"])
    X_test, y_test = te.drop(columns="hair_fall", te["hair_fall"])
    n = test_rows()
    if n and n < len(X_test):
        X_test, y_test = context(X_test, y_test, n)
    print(f"[test set] {len(X_test):,} rows (change in backend/workspace/test_rows.txt)")
    return X_train, X_test, y_train, y_test

def context(X_train, y_train, size):
    """Stratified training subset of `size` rows (None = all). Same rows for every model."""
    if size is None or size >= len(X_train):
        return X_train, y_train
    X_ctx, _, y_ctx, _ = train_test_split(X_train, y_train, train_size=size, stratify=y_train,
                                          random_state=SEED)
    return X_ctx, y_ctx

def device() -> str:
    import torch
    if torch.cuda.is_available():
        return "cuda"
    return "mps" if torch.backends.mps.is_available() else "cpu"

def predict_proba_batched(model, X, batch=500, tag=None):
    """predict_proba in batches with progress + ETA. With `tag`, finished batches are saved to
    backend/workspace/checkpoints/<tag>.npz, so a stopped or crashed run resumes where it left
    off."""
    ckpt = OUT.parent / "checkpoints" / f"{tag}.npz" if tag else None
    done_parts = []
    if ckpt is not None:
        ckpt.parent.mkdir(exist_ok=True)
        if ckpt.exists():
            saved = np.load(ckpt)
            if len(saved) <= len(X):
                done_parts = [saved]
                print(f" resuming: {len(saved):,}/{len(X):,} rows already predicted",
flush=True)
            start = sum(len(p) for p in done_parts)
            t0 = time.time()
            for i in range(start, len(X), batch):
                done_parts.append(model.predict_proba(X.iloc[i:i + batch]))
                done = min(i + batch, len(X))
                rate = (time.time() - t0) / (done - start)
                print(f" predicted {done:,}/{len(X):,} rows | elapsed {time.time() - t0:,.0f}s | "
                    f"ETA {rate * (len(X) - done):,.0f}s", flush=True)
            if ckpt is not None:
                np.save(ckpt, np.vstack(done_parts))
    return np.vstack(done_parts)

def evaluate(model_name, size_label, n_context, y_true, proba, fit_seconds, predict_seconds,
extra=None):
    """Metrics + confusion matrix; saves metrics.json and predictions.csv to outputs/."""
    rows = y_true.index if hasattr(y_true, "index") else np.arange(len(y_true))
    y_true = np.asarray(y_true)
    y_pred = proba.argmax(axis=1)
    m = {
        "model": model_name, "size": size_label, "context_rows": int(n_context), "test_rows":
int(len(y_true)),
        "accuracy": round(accuracy_score(y_true, y_pred), 4),
        "macro_f1": round(f1_score(y_true, y_pred, average="macro"), 4),
        "weighted_f1": round(f1_score(y_true, y_pred, average="weighted"), 4),
        "roc_auc_ovr": round(roc_auc_score(y_true, proba, multi_class="ovr", average="macro"),
4),
        "fit_seconds": round(fit_seconds, 1), "predict_seconds": round(predict_seconds, 1),
        **(extra or {}),

```

```

    }
    print(json.dumps(m, indent=2))
    print(classification_report(y_true, y_pred, target_names=CLASS_NAMES, digits=4))

    fig, ax = plt.subplots(figsize=(4.5, 4))
    ConfusionMatrixDisplay.from_predictions(y_true, y_pred, display_labels=CLASS_NAMES,
cmap="Blues", ax=ax,
                                         colorbar=False)
    ax.set_title(f"{model_name} ({size_label}) - confusion matrix")
    fig.tight_layout()
    fig.savefig(OUT / "confusion_matrix.png", dpi=150)
    plt.show()

    (OUT / "metrics.json").write_text(json.dumps(m, indent=2))
    pd.DataFrame({"row": rows, "y_true": y_true, "y_pred": y_pred, **{f"p_{c}": proba[:, i] for
i, c in enumerate(CLASS_NAMES)}}) \
        .to_csv(OUT / "predictions.csv", index=False)
    return m

def shap_report(model_name, size_label, shap_values, X_explained, seconds, extra=None):
    """Global importance bar + beeswarm (High class); saves shap_importance.csv and
shap_summary.json."""
    import shap
    sv = np.stack(shap_values, axis=-1) if isinstance(shap_values, list) else
np.asarray(shap_values)
    if sv.ndim == 2: # single-output model
        sv = sv[:, :, None]
    feats = list(X_explained.columns)
    imp = pd.DataFrame({"feature": feats, **{f"mean_abs_{c}": np.abs(sv[:, :, i]).mean(0) for
i, c in enumerate(CLASS_NAMES)}})
    imp["mean_abs_all"] = imp[[f"mean_abs_{c}" for c in CLASS_NAMES]].mean(1)
    imp = imp.sort_values("mean_abs_all", ascending=False).reset_index(drop=True)
    imp.to_csv(OUT / "shap_importance.csv", index=False)
    print(imp.round(4).to_string(index=False))

    fig, ax = plt.subplots(figsize=(7, 6))
    top = imp.head(15)[::-1]
    ax.barh(top["feature"], top["mean_abs_all"], color="#2563eb")
    ax.set_xlabel("mean |SHAP| (averaged over classes)")
    ax.set_title(f"{model_name} ({size_label}) - global feature importance")
    fig.tight_layout(); fig.savefig(OUT / "shap_global_importance.png", dpi=150); plt.show()

    plt.figure()
    shap.summary_plot(sv[:, :, 2], X_explained, show=False, max_display=15)
    plt.title(f"{model_name} ({size_label}) - SHAP for High risk")
    plt.tight_layout(); plt.savefig(OUT / "shap_beeswarm_high.png", dpi=150); plt.show()

    summary = {"model": model_name, "size": size_label, "rows_explained":
int(len(X_explained)),
               "seconds": round(seconds, 1), "top_5": imp["feature"].head(5).tolist(), **(extra
or {})}
    (OUT / "shap_summary.json").write_text(json.dumps(summary, indent=2))
    print(json.dumps(summary, indent=2))
    return imp

```

A.2 Step 1 - Environment check

```

# Step 1 - Environment check
# Environment check (all packages are pre-installed in backend/.venv)
import os
import platform
import sys
from importlib.metadata import version

import torch

print("Python ", sys.version.split()[0])
print("System ", platform.platform())
print("RAM      ", round(os.sysconf("SC_PAGE_SIZE") * os.sysconf("SC_PHYS_PAGES") / 1e9, 1),
"GB")
print("Device ", "mps (Apple GPU)" if torch.backends.mps.is_available() else "cpu")
print()

```

```
for pkg in ["pandas", "numpy", "scikit-learn", "statsmodels", "catboost", "tabpfn", "tabfm",
"shap", "torch", "matplotlib"]:
    print(f"{pkg:14s} {version(pkg)}")
```

A.3 Step 2 - Load dataset

```
# Step 2 - Load dataset
# Load the dataset (loaded into the local Postgres automatically at startup)
import matplotlib.pyplot as plt
from lab import CLASS_NAMES, OUT, load_df

df_raw = load_df("hairfall_dataset")
print("Shape:", df_raw.shape)
print("Missing values:", int(df_raw.isna().sum().sum()))
print("\nColumn types:\n" + df_raw.dtypes.to_string())
display(df_raw.head())

counts = df_raw["hair_fall"].value_counts().sort_index()
print("\nTarget distribution:")
for k, v in counts.items():
    print(f" {k} {CLASS_NAMES[k]:9s} {v:6,d} ({v / len(df_raw):.1%})")

fig, ax = plt.subplots(figsize=(5, 3.5))
ax.bar(CLASS_NAMES, counts.values, color=["#16a34a", "#f59e0b", "#dc2626"])
ax.set_title("Hair fall risk - class distribution"); ax.set_ylabel("patients")
fig.tight_layout(); fig.savefig(OUT / "class_distribution.png", dpi=150); plt.show()

df_raw.describe().T.round(3).to_csv(OUT / "data_summary.csv")
```

A.4 Step 3 - Validate & encode

```
# Step 3 - Validate & encode
# Validate against clinical reference ranges and encode
import matplotlib.pyplot as plt
import pandas as pd
from lab import OUT, load_df

df = load_df("hairfall_dataset")
n0 = len(df)

# 1) Sanity checks
assert set(df["gender"].unique()) <= {"Female", "Male", "Other"}, "unexpected gender label"
print("Gender:", df["gender"].value_counts().to_dict())
print("Missing values:", int(df.isna().sum().sum()),
      "\n| Duplicate rows:", int(df.drop(columns=["id"]).duplicated().sum()))

# 2) Clinical reference check: (normal_low, normal_high, impossible_low, impossible_high)
# Abnormal values are KEPT (they are what predicts hair fall); only impossible values are
# dropped.
REF = {
    "total_protein": (6.0, 8.3, 2.0, 12.0), # g/dL serum total protein
    "calcium": (8.5, 10.5, 5.0, 14.0), # mg/dL serum calcium
    "iron": (60, 170, 5, 500), # ug/dL serum iron
    "vitamin_d": (25, 80, 1, 150), # ng/mL 25-OH vitamin D
    "alt_liver": (7, 56, 1, 500), # U/L ALT
    "manganese": (4, 15, 0.5, 60), # ug/L whole-blood manganese
    "body_water_content": (45, 65, 20, 85), # % total body water
}
rows = []
for c, (lo, hi, ilo, ihi) in REF.items():
    below, above = int((df[c] < lo).sum()), int((df[c] > hi).sum())
    rows.append(dict(feature=c, normal_range=f"{lo}-{hi}", below_normal=below,
                    above_normal=above,
                    impossible=int(((df[c] < ilo) | (df[c] > ihi)).sum()),
                    pct_abnormal=round(100 * (below + above) / len(df), 1)))
df = df[(df[c] >= ilo) & (df[c] <= ihi)]
report = pd.DataFrame(rows)
display(report)
print(f"Rows removed as physiologically impossible: {n0 - len(df)}")

fig, ax = plt.subplots(figsize=(6, 3.5))
```

```

ax.barh(report["feature"], report["pct_abnormal"], color="#2563eb")
ax.set_xlabel("% of patients outside the normal range"); ax.set_title("Biomarkers outside
reference ranges")
fig.tight_layout(); fig.savefig(OUT / "range_check.png", dpi=150); plt.show()

# 3) Encode gender, drop identifiers (id, full_name are personal / non-predictive)
df["gender"] = df["gender"].map({"Female": 0, "Male": 1, "Other": 2})
clean = df.drop(columns=["id", "full_name"]).reset_index(drop=True)
assert clean.isna().sum().sum() == 0
print("Clean shape:", clean.shape, "| features:", clean.shape[1] - 1)

clean.to_csv(OUT / "hairfall_clean.csv", index=False)
report.to_csv(OUT / "range_report.csv", index=False)

```

A.5 Step 4 - Train/test split

```

# Step 4 - Train/test split
# 80/20 stratified train/test split + the matched training sizes used in steps 5-8
import pandas as pd
from lab import OUT, SEED, SIZES, context, step_file
from sklearn.model_selection import train_test_split

clean = pd.read_csv(step_file("Step_03_CleanEncode", "hairfall_clean.csv"))
train, test = train_test_split(clean, test_size=0.20, stratify=clean["hair_fall"],
random_state=SEED)
print(f"Train: {len(train):,} rows | Test: {len(test):,} rows")

X_train, y_train = train.drop(columns="hair_fall", train["hair_fall"])
rows = []
for label, n in SIZES.items():
    _, y_ctx = context(X_train, y_train, n)
    dist = y_ctx.value_counts(normalize=True).sort_index().round(3).to_dict()
    rows.append({"size": label, "rows": len(y_ctx), "Low": dist[0], "Moderate": dist[1],
"High": dist[2]})
sizes = pd.DataFrame(rows)
print("\nTraining sizes (same stratified rows for every model):")
display(sizes)
print("Test class share:",
test["hair_fall"].value_counts(normalize=True).sort_index().round(3).to_dict())

train.to_csv(OUT / "train.csv", index=False)
test.to_csv(OUT / "test.csv", index=False)
sizes.to_csv(OUT / "training_sizes.csv", index=False)

```

A.6 Step 5a - CatBoost · 500

```

# Step 5a - CatBoost · 500
# CatBoost | training size = 500
import itertools
import time

import numpy as np
import pandas as pd
from catboost import CatBoostClassifier
from lab import OUT, SEED, SIZES, context, evaluate, load_split
from sklearn.metrics import f1_score
from sklearn.model_selection import StratifiedKFold

SIZE = "500"
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Training rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

# 1) Grid search: depth x learning rate, 5-fold stratified CV on the training rows (test set
untouched)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
grid, t0 = [], time.time()
for depth, lr in itertools.product([4, 6, 8], [0.01, 0.03, 0.1]):
    f1s, iters = [], []
    for tr, va in skf.split(X_ctx, y_ctx):
        m = CatBoostClassifier(iterations=1000, depth=depth, learning_rate=lr, l2_leaf_reg=3,

```

```

        loss_function="MultiClass", early_stopping_rounds=50,
        random_seed=SEED, verbose=0)
        m.fit(X_ctx.iloc[tr], y_ctx.iloc[tr], eval_set=(X_ctx.iloc[va], y_ctx.iloc[va]))
        f1s.append(f1_score(y_ctx.iloc[va], m.predict(X_ctx.iloc[va]).ravel(),
        average="macro"))
        iters.append(m.get_best_iteration() + 1)
        grid.append({"depth": depth, "learning_rate": lr, "cv_macro_f1": round(np.mean(f1s), 4),
        "cv_std": round(np.std(f1s), 4), "iterations": int(np.mean(iters))})
        print(f"depth={depth} lr={lr:<5} CV macro-F1 = {np.mean(f1s):.4f} +/- {np.std(f1s):.4f}",
        flush=True)
    grid = pd.DataFrame(grid).sort_values("cv_macro_f1", ascending=False).reset_index(drop=True)
    grid.to_csv(OUT / "grid_search.csv", index=False)
    best = grid.iloc[0]
    print(f"\nBest: depth={int(best.depth)} lr={best.learning_rate}
    iterations={int(best.iterations)} "
        f"(CV macro-F1 {best.cv_macro_f1:.4f}) | grid search took {time.time() - t0:,.0f}s")

# 2) Final model on all training rows with the best parameters
t = time.time()
model = CatBoostClassifier(iterations=int(best.iterations), depth=int(best.depth),
        learning_rate=float(best.learning_rate),
        l2_leaf_reg=3, loss_function="MultiClass", random_seed=SEED,
        verbose=0)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

# 3) Test
t = time.time()
proba = model.predict_proba(X_test)
pred_s = time.time() - t
model.save_model(str(OUT / "catboost.cbm"))
evaluate("CatBoost", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s,
        extra={"depth": int(best.depth), "learning_rate": float(best.learning_rate),
        "iterations": int(best.iterations), "cv_macro_f1": float(best.cv_macro_f1)})

```

A.7 Step 5b - CatBoost · 2000

```

# Step 5b - CatBoost · 2000
# CatBoost | training size = 2000
import itertools
import time

import numpy as np
import pandas as pd
from catboost import CatBoostClassifier
from lab import OUT, SEED, SIZES, context, evaluate, load_split
from sklearn.metrics import f1_score
from sklearn.model_selection import StratifiedKFold

SIZE = "2000"
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Training rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

# 1) Grid search: depth x learning rate, 5-fold stratified CV on the training rows (test set
untouched)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
grid, t0 = [], time.time()
for depth, lr in itertools.product([4, 6, 8], [0.01, 0.03, 0.1]):
    f1s, iters = [], []
    for tr, va in skf.split(X_ctx, y_ctx):
        m = CatBoostClassifier(iterations=1000, depth=depth, learning_rate=lr, l2_leaf_reg=3,
        loss_function="MultiClass", early_stopping_rounds=50,
        random_seed=SEED, verbose=0)
        m.fit(X_ctx.iloc[tr], y_ctx.iloc[tr], eval_set=(X_ctx.iloc[va], y_ctx.iloc[va]))
        f1s.append(f1_score(y_ctx.iloc[va], m.predict(X_ctx.iloc[va]).ravel(),
        average="macro"))
        iters.append(m.get_best_iteration() + 1)
    grid.append({"depth": depth, "learning_rate": lr, "cv_macro_f1": round(np.mean(f1s), 4),
    "cv_std": round(np.std(f1s), 4), "iterations": int(np.mean(iters))})
    print(f"depth={depth} lr={lr:<5} CV macro-F1 = {np.mean(f1s):.4f} +/- {np.std(f1s):.4f}",
    flush=True)
grid = pd.DataFrame(grid).sort_values("cv_macro_f1", ascending=False).reset_index(drop=True)
grid.to_csv(OUT / "grid_search.csv", index=False)
best = grid.iloc[0]

```

```

print(f"\nBest: depth={int(best.depth)} lr={best.learning_rate}
iterations={int(best.iterations)} "
      f"(CV macro-F1 {best.cv_macro_f1:.4f}) | grid search took {time.time() - t0:,.0f}s")

# 2) Final model on all training rows with the best parameters
t = time.time()
model = CatBoostClassifier(iterations=int(best.iterations), depth=int(best.depth),
learning_rate=float(best.learning_rate),
                        l2_leaf_reg=3, loss_function="MultiClass", random_seed=SEED,
verbose=0)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

# 3) Test
t = time.time()
proba = model.predict_proba(X_test)
pred_s = time.time() - t
model.save_model(str(OUT / "catboost.cbm"))
evaluate("CatBoost", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s,
        extra={"depth": int(best.depth), "learning_rate": float(best.learning_rate),
"iterations": int(best.iterations), "cv_macro_f1": float(best.cv_macro_f1)})

```

A.8 Step 5c - CatBoost · Full

```

# Step 5c - CatBoost · Full
# CatBoost | training size = Full
import itertools
import time

import numpy as np
import pandas as pd
from catboost import CatBoostClassifier
from lab import OUT, SEED, SIZES, context, evaluate, load_split
from sklearn.metrics import f1_score
from sklearn.model_selection import StratifiedKFold

SIZE = "Full"
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Training rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

# 1) Grid search: depth x learning rate, 5-fold stratified CV on the training rows (test set
untouched)
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=SEED)
grid, t0 = [], time.time()
for depth, lr in itertools.product([4, 6, 8], [0.01, 0.03, 0.1]):
    f1s, iters = [], []
    for tr, va in skf.split(X_ctx, y_ctx):
        m = CatBoostClassifier(iterations=1000, depth=depth, learning_rate=lr, l2_leaf_reg=3,
loss_function="MultiClass", early_stopping_rounds=50,
random_seed=SEED, verbose=0)
        m.fit(X_ctx.iloc[tr], y_ctx.iloc[tr], eval_set=(X_ctx.iloc[va], y_ctx.iloc[va]))
        f1s.append(f1_score(y_ctx.iloc[va], m.predict(X_ctx.iloc[va]).ravel(),
average="macro"))
        iters.append(m.get_best_iteration() + 1)
    grid.append({"depth": depth, "learning_rate": lr, "cv_macro_f1": round(np.mean(f1s), 4),
"cv_std": round(np.std(f1s), 4), "iterations": int(np.mean(iters))})
    print(f"depth={depth} lr={lr:<5} CV macro-F1 = {np.mean(f1s):.4f} +/- {np.std(f1s):.4f}",
flush=True)
grid = pd.DataFrame(grid).sort_values("cv_macro_f1", ascending=False).reset_index(drop=True)
grid.to_csv(OUT / "grid_search.csv", index=False)
best = grid.iloc[0]
print(f"\nBest: depth={int(best.depth)} lr={best.learning_rate}
iterations={int(best.iterations)} "
      f"(CV macro-F1 {best.cv_macro_f1:.4f}) | grid search took {time.time() - t0:,.0f}s")

# 2) Final model on all training rows with the best parameters
t = time.time()
model = CatBoostClassifier(iterations=int(best.iterations), depth=int(best.depth),
learning_rate=float(best.learning_rate),
                        l2_leaf_reg=3, loss_function="MultiClass", random_seed=SEED,
verbose=0)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

```

```
# 3) Test
t = time.time()
proba = model.predict_proba(X_test)
pred_s = time.time() - t
model.save_model(str(OUT / "catboost.cbm"))
evaluate("CatBoost", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s,
        extra={"depth": int(best.depth), "learning_rate": float(best.learning_rate),
               "iterations": int(best.iterations), "cv_macro_f1": float(best.cv_macro_f1)})
```

A.9 Step 6a - TabPFN · 500

```
# Step 6a - TabPFN · 500
# TabPFN (pretrained foundation model, no training - rows are given as context) | context = 500
import time

from lab import SEED, SIZES, context, device, evaluate, load_split, predict_proba_batched
from tabpfn import TabPFNClassifier

SIZE = "500"
BATCH = 500 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
dev = device()
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,} | device: {dev}")

t = time.time()
model = TabPFNClassifier(device=dev, random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabpfn_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabPFN", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s, extra={"device": dev})
```

A.10 Step 6b - TabPFN · 2000

```
# Step 6b - TabPFN · 2000
# TabPFN (pretrained foundation model, no training - rows are given as context) | context = 2000
import time

from lab import SEED, SIZES, context, device, evaluate, load_split, predict_proba_batched
from tabpfn import TabPFNClassifier

SIZE = "2000"
BATCH = 500 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
dev = device()
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,} | device: {dev}")

t = time.time()
model = TabPFNClassifier(device=dev, random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabpfn_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabPFN", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s, extra={"device": dev})
```

A.11 Step 6c - TabPFN · Full

```
# Step 6c - TabPFN · Full
```

```

# TabPFN (pretrained foundation model, no training - rows are given as context) | context =
Full
import time

from lab import SEED, SIZES, context, device, evaluate, load_split, predict_proba_batched
from tabPFN import TabPFNCClassifier

SIZE = "Full"
BATCH = 500 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
dev = device()
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,} | device: {dev}")

t = time.time()
model = TabPFNCClassifier(device=dev, random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabPFN_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabPFN", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s, extra={"device": dev})

```

A.12 Step 7a - TabFM · 500

```

# Step 7a - TabFM · 500
# TabFM (Google foundation model, rows are given as context) | context = 500
import time

from lab import SIZES, context, evaluate, load_split, predict_proba_batched
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "500"
BATCH = 100 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

t = time.time()
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabfm_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabFM", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s)

```

A.13 Step 7b - TabFM · 2000

```

# Step 7b - TabFM · 2000
# TabFM (Google foundation model, rows are given as context) | context = 2000
import time

from lab import SIZES, context, evaluate, load_split, predict_proba_batched
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "2000"
BATCH = 100 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

t = time.time()
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)

```

```

fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabfm_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabFM", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s)

```

A.14 Step 7c - TabFM · Full

```

# Step 7c - TabFM · Full
# TabFM (Google foundation model, rows are given as context) | context = Full
import time

from lab import SIZES, context, evaluate, load_split, predict_proba_batched
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "Full"
BATCH = 100 # test rows per prediction batch - lower this if the Mac runs out of memory
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
print(f"Context rows: {len(X_ctx):,} | Test rows: {len(X_test):,}")

t = time.time()
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)
fit_s = time.time() - t

t = time.time()
proba = predict_proba_batched(model, X_test, BATCH, tag=f"tabfm_{SIZE}_{len(X_test)}")
pred_s = time.time() - t
evaluate("TabFM", SIZE, len(X_ctx), y_test, proba, fit_s, pred_s)

```

A.15 Step 8a - CatBoost SHAP · 500

```

# Step 8a - CatBoost SHAP · 500
# CatBoost SHAP (TreeExplainer, exact) | model from Step 5a
import time

import shap
from catboost import CatBoostClassifier
from lab import SEED, load_split, shap_report, step_file

SIZE = "500"
N_EXPLAIN = 1000 # test rows to explain
X_train, X_test, y_train, y_test = load_split()
model = CatBoostClassifier()
model.load_model(str(step_file("Step_05a_CatBoost_500", "catboost.cbm")))
X_exp = X_test.sample(min(N_EXPLAIN, len(X_test)), random_state=SEED)

t = time.time()
shap_values = shap.TreeExplainer(model).shap_values(X_exp)
shap_report("CatBoost", SIZE, shap_values, X_exp, time.time() - t, extra={"explainer":
"TreeExplainer"})

```

A.16 Step 8b - CatBoost SHAP · 2000

```

# Step 8b - CatBoost SHAP · 2000
# CatBoost SHAP (TreeExplainer, exact) | model from Step 5b
import time

import shap
from catboost import CatBoostClassifier
from lab import SEED, load_split, shap_report, step_file

SIZE = "2000"

```

```

N_EXPLAIN = 1000 # test rows to explain
X_train, X_test, y_train, y_test = load_split()
model = CatBoostClassifier()
model.load_model(str(step_file("Step_05b_CatBoost_2000", "catboost.cbm")))
X_exp = X_test.sample(min(N_EXPLAIN, len(X_test)), random_state=SEED)

t = time.time()
shap_values = shap.TreeExplainer(model).shap_values(X_exp)
shap_report("CatBoost", SIZE, shap_values, X_exp, time.time() - t, extra={"explainer":
"TreeExplainer"})

```

A.17 Step 8c - CatBoost SHAP · Full

```

# Step 8c - CatBoost SHAP · Full
# CatBoost SHAP (TreeExplainer, exact) | model from Step 5c
import time

import shap
from catboost import CatBoostClassifier
from lab import SEED, load_split, shap_report, step_file

SIZE = "Full"
N_EXPLAIN = 1000 # test rows to explain
X_train, X_test, y_train, y_test = load_split()
model = CatBoostClassifier()
model.load_model(str(step_file("Step_05c_CatBoost_Full", "catboost.cbm")))
X_exp = X_test.sample(min(N_EXPLAIN, len(X_test)), random_state=SEED)

t = time.time()
shap_values = shap.TreeExplainer(model).shap_values(X_exp)
shap_report("CatBoost", SIZE, shap_values, X_exp, time.time() - t, extra={"explainer":
"TreeExplainer"})

```

A.18 Step 8d - TabPFN SHAP · 500

```

# Step 8d - TabPFN SHAP · 500
# TabPFN SHAP (KernelExplainer, model-agnostic) | context = 500
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report
import pandas as pd
from lab import device
from tabPFN import TabPFNClassifier

SIZE = "500"
N_EXPLAIN = 30 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 10 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabPFNClassifier(device=device(), random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()

```

```

shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabPFN", SIZE, shap_values, X_exp, time.time() - t,
            extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.19 Step 8e - TabPFN SHAP · 2000

```

# Step 8e - TabPFN SHAP · 2000
# TabPFN SHAP (KernelExplainer, model-agnostic) | context = 2000
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report
import pandas as pd
from lab import device
from tabPFN import TabPFNClassifier

SIZE = "2000"
N_EXPLAIN = 30 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 10 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabPFNClassifier(device=device(), random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()
shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabPFN", SIZE, shap_values, X_exp, time.time() - t,
            extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.20 Step 8f - TabPFN SHAP · Full

```

# Step 8f - TabPFN SHAP · Full
# TabPFN SHAP (KernelExplainer, model-agnostic) | context = Full
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report
import pandas as pd
from lab import device
from tabPFN import TabPFNClassifier

SIZE = "Full"
N_EXPLAIN = 30 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 10 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabPFNClassifier(device=device(), random_state=SEED, ignore_pretraining_limits=True)
model.fit(X_ctx, y_ctx)

```

```

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()
shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabPFN", SIZE, shap_values, X_exp, time.time() - t,
            extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.21 Step 8g - TabFM SHAP · 500

```

# Step 8g - TabFM SHAP · 500
# TabFM SHAP (KernelExplainer, model-agnostic) | context = 500
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report
import pandas as pd
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "500"
N_EXPLAIN = 20 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 5 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()
shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabFM", SIZE, shap_values, X_exp, time.time() - t,
            extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.22 Step 8h - TabFM SHAP · 2000

```

# Step 8h - TabFM SHAP · 2000
# TabFM SHAP (KernelExplainer, model-agnostic) | context = 2000
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report

```

```

import pandas as pd
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "2000"
N_EXPLAIN = 20 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 5 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()
shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabFM", SIZE, shap_values, X_exp, time.time() - t,
extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.23 Step 8i - TabFM SHAP · Full

```

# Step 8i - TabFM SHAP · Full
# TabFM SHAP (KernelExplainer, model-agnostic) | context = Full
import time

import numpy as np
import shap
from lab import SEED, SIZES, context, load_split, shap_report
import pandas as pd
from tabfm import TabFMClassifier
from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1

SIZE = "Full"
N_EXPLAIN = 20 # test rows to explain (cost ~ N_EXPLAIN x NSAMPLES x BACKGROUND predictions)
NSAMPLES = 100
BACKGROUND = 5 # k-means summary of the context used as SHAP background
X_train, X_test, y_train, y_test = load_split()
X_ctx, y_ctx = context(X_train, y_train, SIZES[SIZE])
model = TabFMClassifier(model=tabfm_v1.load(), n_estimators=1)
model.fit(X_ctx, y_ctx)

def predict_proba(a):
    X = pd.DataFrame(a, columns=X_ctx.columns)
    if len(X) == 1: # some foundation models fail on a single row: pad and keep the first
        return model.predict_proba(pd.concat([X, X]))[:1]
    return model.predict_proba(X)

X_exp = X_test.sample(N_EXPLAIN, random_state=SEED)
background = shap.kmeans(X_ctx, BACKGROUND)
print(f"Explaining {N_EXPLAIN} rows, {NSAMPLES} samples each, {BACKGROUND} background
points...", flush=True)
t = time.time()
shap_values = shap.KernelExplainer(predict_proba, background).shap_values(X_exp,
nsamples=NSAMPLES)
shap_report("TabFM", SIZE, shap_values, X_exp, time.time() - t,
extra={"explainer": "KernelExplainer", "nsamples": NSAMPLES, "background":
BACKGROUND})

```

A.24 Step 9 - McNemar's tests

```
# Step 9 - McNemar's tests
# McNemar's test: are the models' error patterns significantly different? (per training size)
import itertools

import pandas as pd
from lab import OUT, PROJECT
from statsmodels.stats.contingency_tables import mcnemar

MODELS = {"CatBoost": "Step_05", "TabPFN": "Step_06", "TabFM": "Step_07"}
LETTERS = {"500": "a", "2000": "b", "Full": "c"}

rows = []
for size, letter in LETTERS.items():
    preds = {}
    for name, prefix in MODELS.items():
        f = PROJECT / f"{prefix}{letter}_{name}_{size}" / "predictions.csv"
        if f.exists():
            d = pd.read_csv(f)
            preds[name] = d.set_index("row").assign(correct=lambda x: x.y_true == x.y_pred)
    ["correct"]
    else:
        print(f"[skip] {name} {size}: not trained yet")
    for a, b in itertools.combinations(preds, 2):
        common = preds[a].index.intersection(preds[b].index) # same test rows only
        ca, cb = preds[a].loc[common], preds[b].loc[common]
        table = [[int((ca & cb).sum()), int((ca & ~cb).sum()), [int((~ca & cb).sum()),
int((~ca & ~cb).sum())]]
        res = mcnemar(table, exact=False, correction=True)
        rows.append({"size": size, "comparison": f"{a} vs {b}", "test_rows": len(common),
f"only_{a}_correct": table[0][1], "only_other_correct": table[1][0],
"chi2": round(float(res.statistic), 4), "p_value":
round(float(res.pvalue), 4),
"significant_0.05": bool(res.pvalue < 0.05)})

results = pd.DataFrame(rows)
display(results)
results.to_csv(OUT / "mcnemar_results.csv", index=False)
```

A.25 Step 10 - Final results

```
# Step 10 - Final results
# Final comparison: all models x all training sizes, scaling curves and timings
import json

import matplotlib.pyplot as plt
import pandas as pd
from lab import OUT, PROJECT

metrics = [json.loads(p.read_text()) for p in
sorted(PROJECT.glob("Step_0[567]*metrics.json"))]
if not metrics:
    raise RuntimeError("No trained models found - run steps 5-7 first.")
res = pd.DataFrame(metrics)
order = {"500": 0, "2000": 1, "Full": 2}
res = res.sort_values(["size", "model"], key=lambda s: s.map(order) if s.name == "size" else
s).reset_index(drop=True)
cols = ["model", "size", "context_rows", "test_rows", "accuracy", "macro_f1", "weighted_f1",
"roc_auc_ovr",
"fit_seconds", "predict_seconds"]
display(res[cols])
res[cols].to_csv(OUT / "final_results.csv", index=False)

shap_rows = [json.loads(p.read_text()) for p in
sorted(PROJECT.glob("Step_08*/shap_summary.json"))]
if shap_rows:
    shap_df = pd.DataFrame(shap_rows)
    shap_df["top_5"] = shap_df["top_5"].str.join(", ")
    display(shap_df)
```

```

shap_df.to_csv(OUT / "shap_top_features.csv", index=False)

colors = {"CatBoost": "#f59e0b", "TabPFN": "#2563eb", "TabFM": "#16a34a"}
fig, axes = plt.subplots(1, 3, figsize=(15, 4.2))
for metric, ax in zip(["accuracy", "macro_f1", "roc_auc_ovr"], axes):
    for model, d in res.groupby("model"):
        ax.plot(d["context_rows"], d[metric], marker="o", label=model, color=colors.get(model))
        ax.set_xscale("log"); ax.set_xlabel("training / context rows (log)"); ax.set_title(metric)
    ax.grid(alpha=.3)
axes[0].legend()
fig.suptitle("Performance vs training size"); fig.tight_layout()
fig.savefig(OUT / "scaling_curves.png", dpi=150); plt.show()

res["total_seconds"] = res["fit_seconds"] + res["predict_seconds"]
pivot = res.pivot(index="size", columns="model", values="total_seconds").reindex(list(order))
ax = pivot.plot(figsize=(7, 4), color=[colors.get(c) for c in pivot.columns], logy=True,
rot=0)
ax.set_ylabel("seconds (log)"); ax.set_title("Train + predict time on this Mac")
plt.tight_layout(); plt.savefig(OUT / "timings.png", dpi=150); plt.show()

```

A.26 Memory and reliability settings added in the Kaggle runs

The steps run on Kaggle are packaged in notebook files (folder backend/kaggle). Each file embeds the module lab.py and the step code above without change, and adds the following block for the foundation models. It splits the prediction into batches that are halved after a GPU out-of-memory error, evaluates the attention in chunks and reduces the column chunk size of TabFM. These settings change the memory use and not the computation.

```

def _chunk_predict(cls, chunk):
    orig = cls.predict_proba

    def predict_proba(self, X, *a, **k):
        parts, i, b = [], 0, chunk
        while i < len(X):
            oom = False
            try:
                parts.append(orig(self, X.iloc[i:i + b] if hasattr(X, "iloc") else X[i:i + b],
*a, **k))
            except torch.cuda.OutOfMemoryError:
                oom = True
                if oom:
                    gc.collect()
                    torch.cuda.empty_cache()
                    if b == 1:
                        raise RuntimeError("GPU out of memory even at batch size 1 - restart the
session and run this step alone")
                    b = max(1, b // 2)
                    print(f" GPU out of memory -> batch size {b}", flush=True)
                    continue
                i += b
        return np.vstack(parts)

    cls.predict_proba = predict_proba

if any(n.startswith("tabpfn") for n in NEEDS):
    from tabpfn import TabPFNClassifier
    _chunk_predict(TabPFNClassifier, 500)

if any(n.startswith("tabfm") for n in NEEDS):
    from tabfm import TabFMClassifier
    from tabfm import tabfm_v1_0_0_pytorch as tabfm_v1
    _chunk_predict(TabFMClassifier, 100)
    _sdpa0 = getattr(torch.nn.functional, "_orig_sdpa", None) or
torch.nn.functional.scaled_dot_product_attention
    torch.nn.functional._orig_sdpa = _sdpa0

```

```

def _chunked_sdpa(q, k, v, attn_mask=None, _f=_sdpa0, **kw):
    n, Q = q.shape[-2], 1024
    if n <= Q:
        return _f(q, k, v, attn_mask=attn_mask, **kw)
    out = []
    for s in range(0, n, Q):
        m = attn_mask[..., s:s + Q, :] if attn_mask is not None and attn_mask.shape[-2] ==
n else attn_mask
        out.append(_f(q[..., s:s + Q, :], k, v, attn_mask=m, **kw))
    return torch.cat(out, dim=-2)

torch.nn.functional.scaled_dot_product_attention = _chunked_sdpa
_tabfm_load = tabfm_v1.load

def _load(*a, _l=_tabfm_load, **k):
    k.setdefault("device", "cuda")
    net = _l(*a, **k)
    for mod in net.modules():
        if hasattr(mod, "col_chunk_size"):
            mod.col_chunk_size = 4
    return net

tabfm_v1.load = _load

```

APPENDIX B

ADDITIONAL FIGURES

This appendix gives the figures that were not needed in the main text: the confusion matrices, the global SHAP importance plots and the SHAP beeswarm plots for the training sizes of 500 and 2,000 rows, and two figures that describe the data.

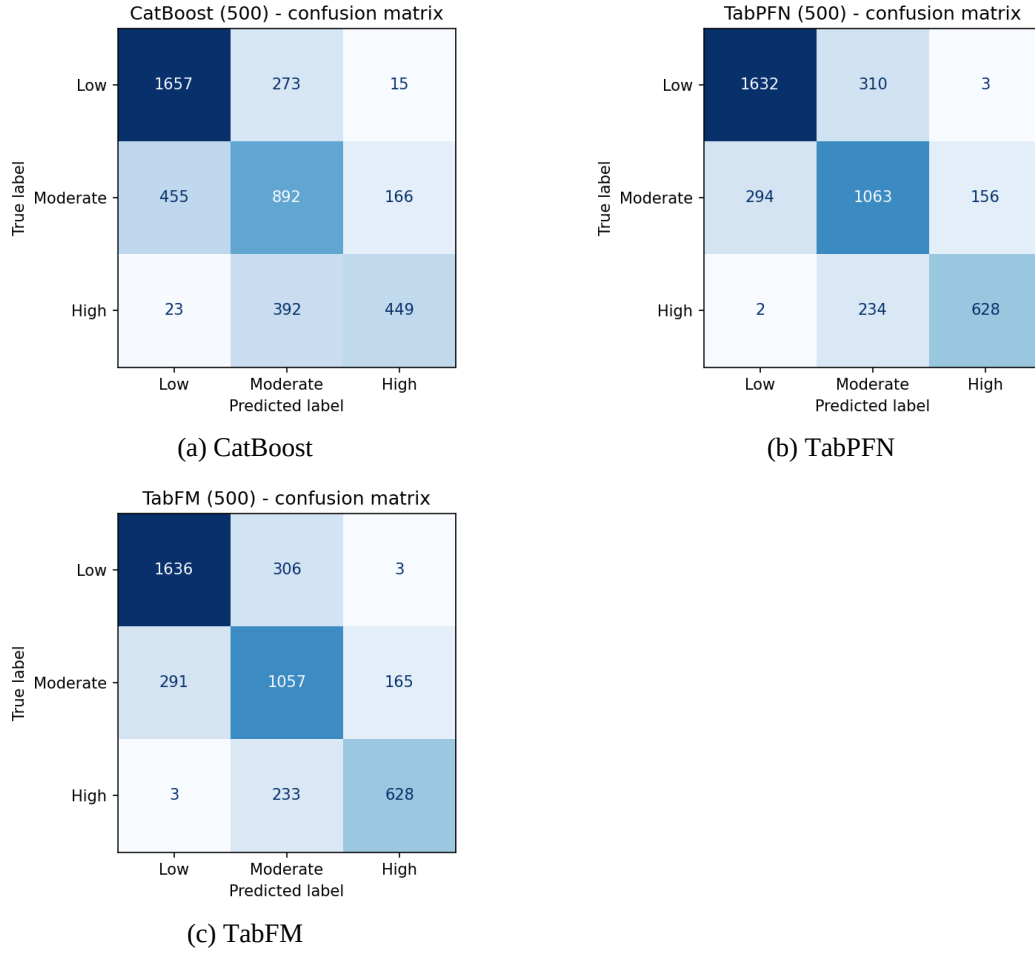
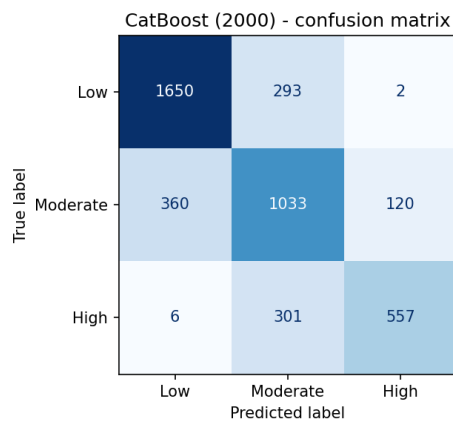
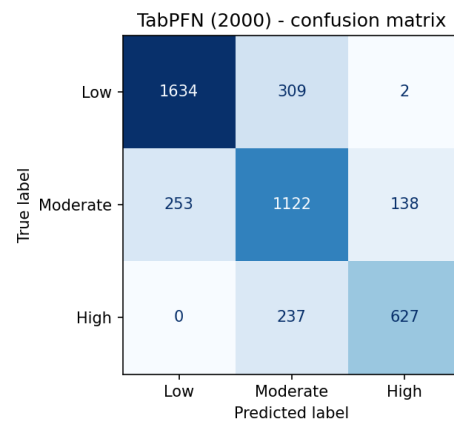


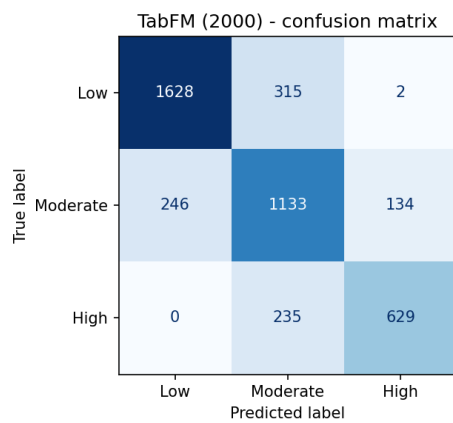
Fig. B.1: Confusion matrices with 500 training rows



(a) CatBoost

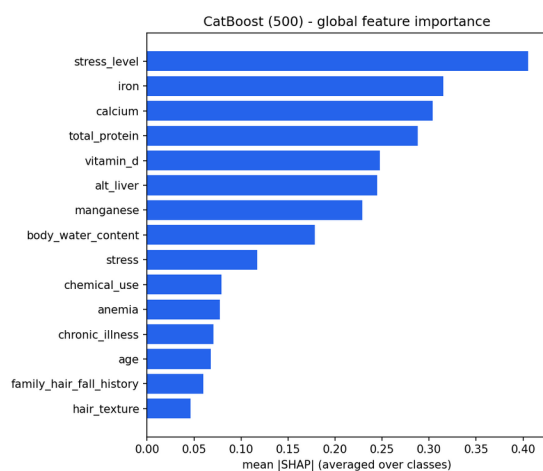


(b) TabPFN

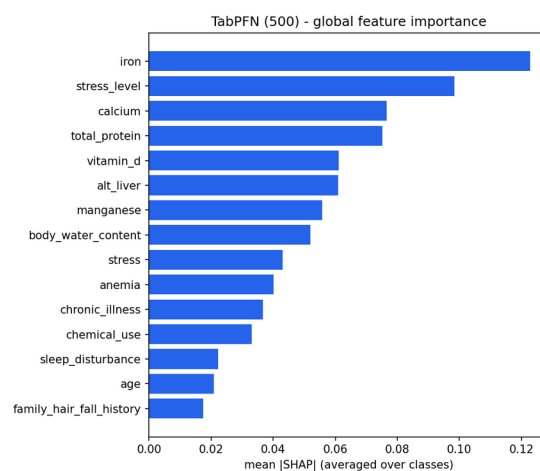


(c) TabFM

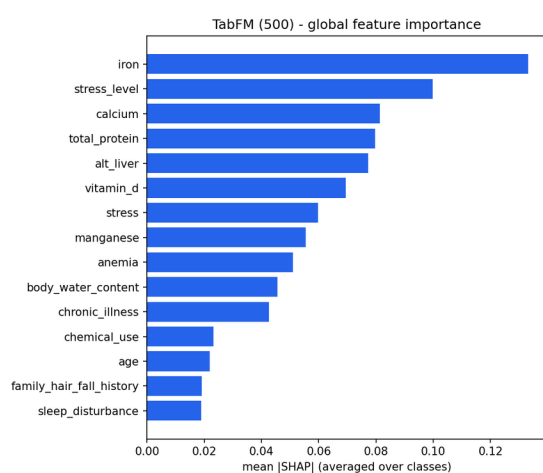
Fig. B.2: Confusion matrices with 2,000 training rows



(a) CatBoost

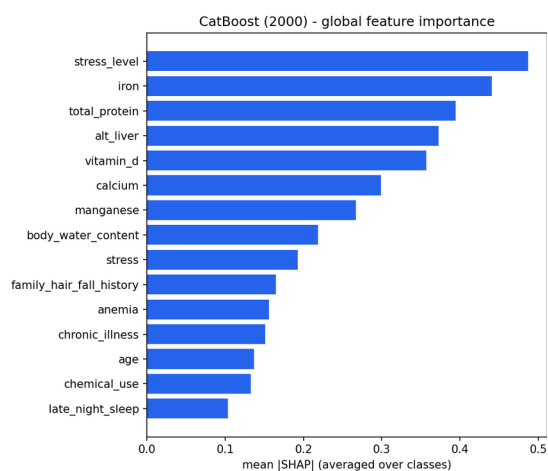


(b) TabPFN

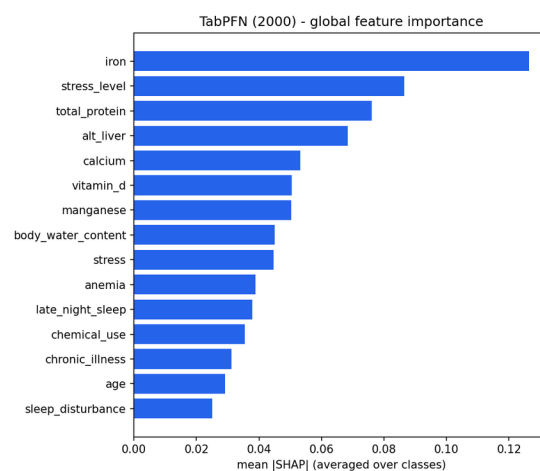


(c) TabFM

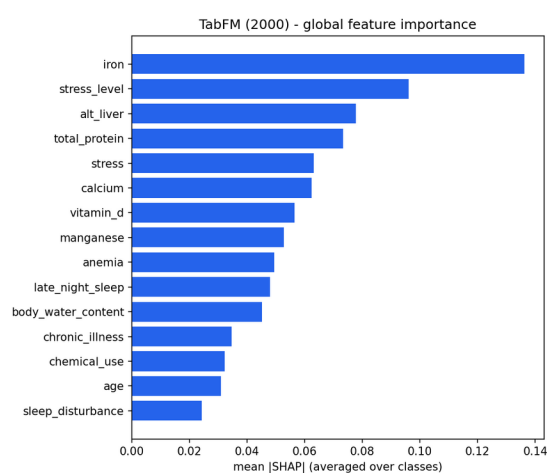
Fig. B.3: Global feature importance (SHAP) with 500 training rows



(a) CatBoost



(b) TabPFN



(c) TabFM

Fig. B.4: Global feature importance (SHAP) with 2,000 training rows

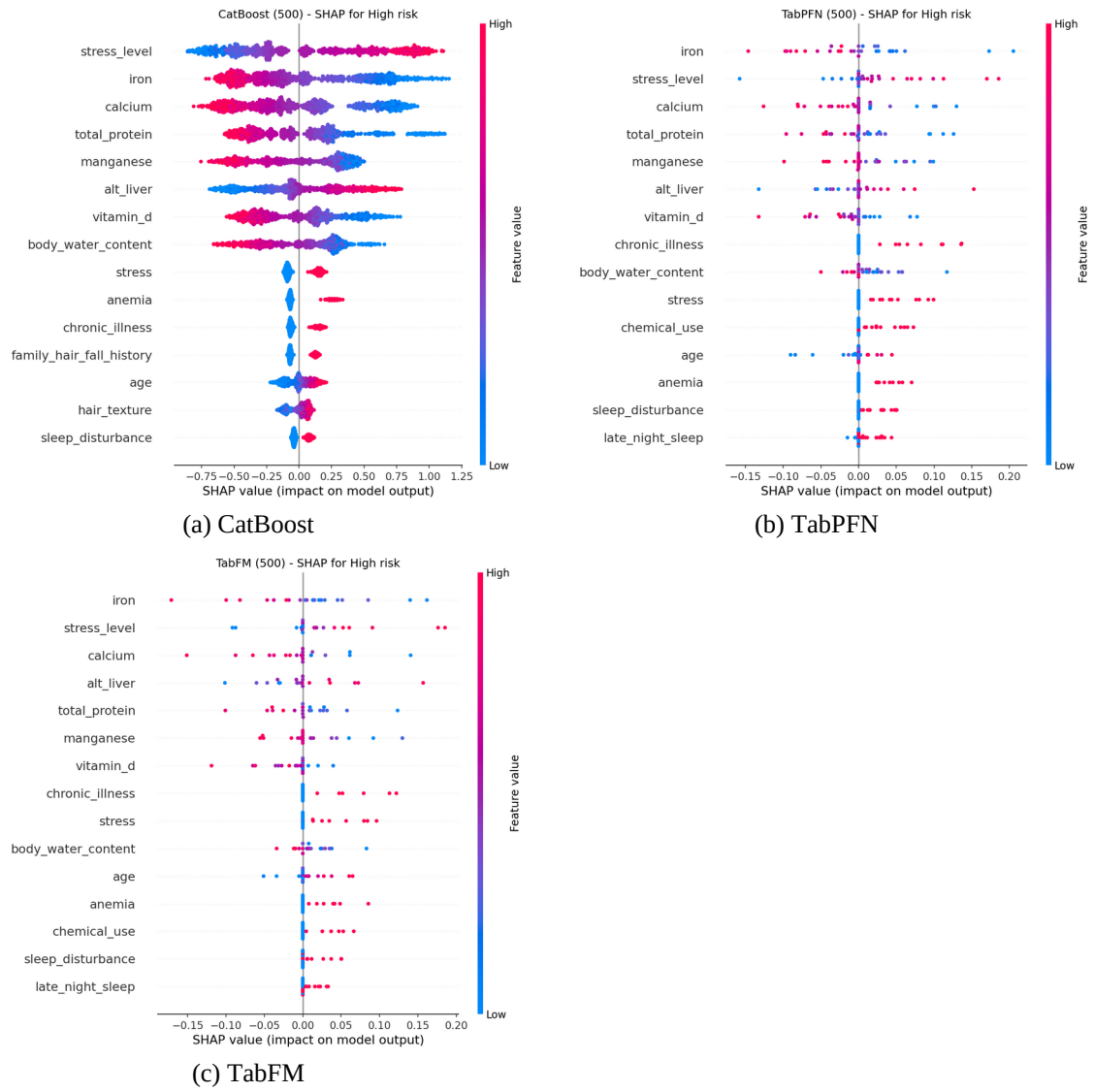


Fig. B.5: SHAP beeswarm plots for the High risk class with 500 training rows

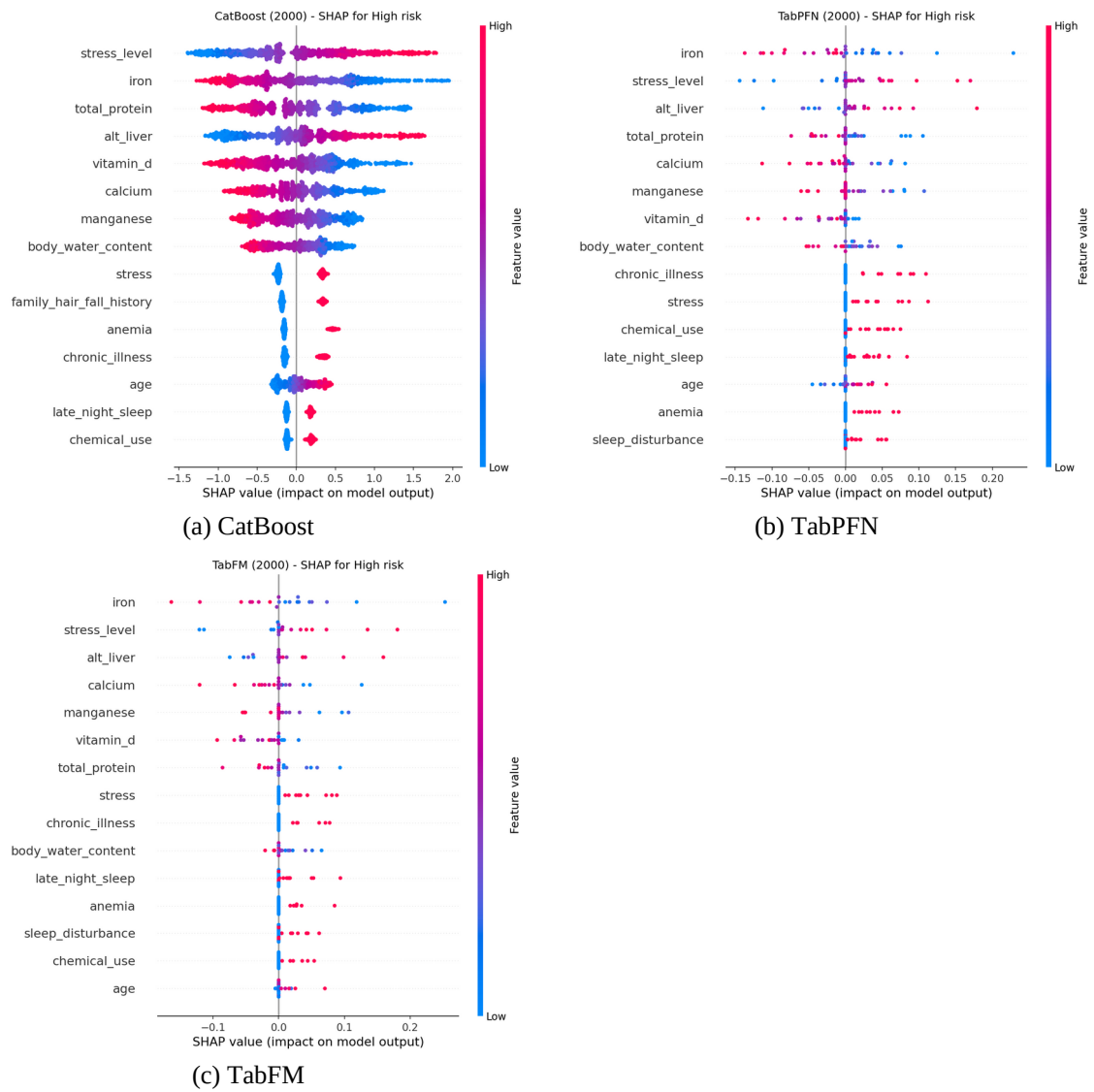


Fig. B.6: SHAP beeswarm plots for the High risk class with 2,000 training rows

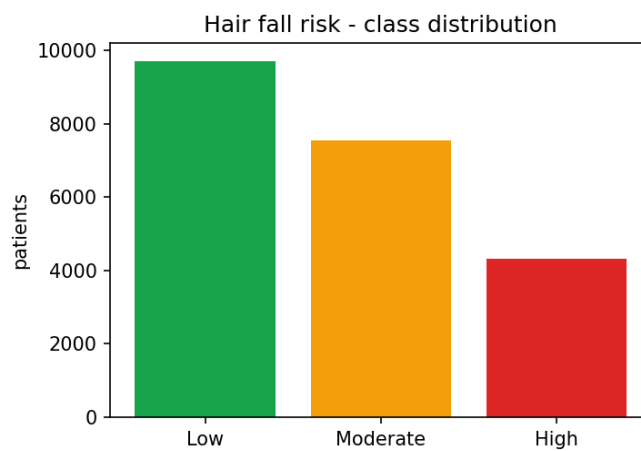


Fig. B.7: Class distribution of the hair fall risk tiers in the dataset

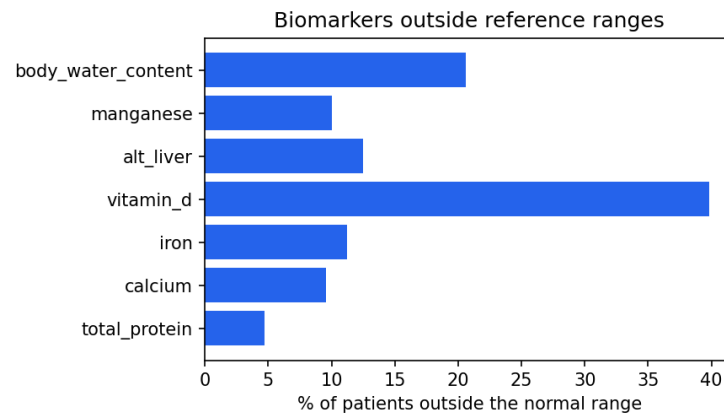


Fig. B.8: Share of records outside the clinical reference range, by biomarker

APPENDIX C

RESULT TABLES FROM THE EXPERIMENTS

Table C.1: Results of all model runs (final_results.csv from Step 10)

Model	Training rows	Accuracy	Macro-F1	Weighted F1	ROC-AUC	Fit (s)	Predict (s)
CatBoost	500	0.6937	0.6648	0.6891	0.8611	0.9	0.0
TabFM	500	0.7684	0.7603	0.7693	0.9151	15.1	0.0
TabPFN	500	0.7689	0.7616	0.7699	0.9159	3.0	0.0
CatBoost	2,000	0.7497	0.7377	0.7496	0.9029	1.9	0.0
TabFM	2,000	0.7844	0.7779	0.7863	0.9209	15.9	209.3
TabPFN	2,000	0.7827	0.7757	0.7844	0.9220	2.3	0.0
CatBoost	17,284	0.7820	0.7744	0.7832	0.9197	3.6	0.0
TabFM	17,284	0.7867	0.7802	0.7882	0.9233	15.1	3,082.2
TabPFN	17,284	0.7862	0.7785	0.7875	0.9235	2.2	0.0

Rows for a run resumed from a checkpoint (TabFM, 500 rows) show a prediction time of 0.

Table C.2: CatBoost grid search with 500 training rows (5-fold cross-validation, sorted by macro-F1)

Depth	Learning rate	CV macro-F1	CV standard deviation	Iterations
4	0.03	0.6589	0.0341	654
4	0.1	0.6408	0.0593	307
6	0.03	0.6372	0.0558	669
6	0.1	0.6339	0.0580	204
4	0.01	0.6289	0.0381	999
8	0.1	0.6225	0.0447	364
6	0.01	0.6195	0.0453	998
8	0.03	0.6184	0.0435	854
8	0.01	0.5935	0.0523	999

Table C.3: CatBoost grid search with 2,000 training rows (5-fold cross-validation, sorted by macro-F1)

Depth	Learning rate	CV macro-F1	CV standard deviation	Iterations
4	0.03	0.7403	0.0126	999
4	0.1	0.7378	0.0157	489
6	0.03	0.7268	0.0103	997
6	0.1	0.7213	0.0210	374
8	0.1	0.7097	0.0085	408
4	0.01	0.7076	0.0267	1000
8	0.03	0.7071	0.0172	999
6	0.01	0.7066	0.0203	1000
8	0.01	0.6888	0.0157	1000

Table C.4: CatBoost grid search with 17,284 training rows (5-fold cross-validation, sorted by macro-F1)

Depth	Learning rate	CV macro-F1	CV standard deviation	Iterations
4	0.1	0.7669	0.0052	807
6	0.03	0.7655	0.0055	1000
6	0.1	0.7648	0.0024	558
4	0.03	0.7628	0.0046	1000
8	0.03	0.7609	0.0061	999
8	0.1	0.7556	0.0046	411
8	0.01	0.7479	0.0105	1000
6	0.01	0.7444	0.0108	1000
4	0.01	0.7310	0.0110	1000

Table C.5: Precision, recall and F1-score by risk tier with 500 and 2,000 training rows

Training rows	Model	Tier	Precision	Recall	F1-score
500	CatBoost	Low	0.7761	0.8519	0.8123
		Moderate	0.5729	0.5896	0.5811
		High	0.7127	0.5197	0.6011
	TabPFN	Low	0.8465	0.8391	0.8428
		Moderate	0.6615	0.7026	0.6814
		High	0.7980	0.7269	0.7608
	TabFM	Low	0.8477	0.8411	0.8444
		Moderate	0.6623	0.6986	0.6800
		High	0.7889	0.7269	0.7566
2,000	CatBoost	Low	0.8185	0.8483	0.8331
		Moderate	0.6349	0.6827	0.6580
		High	0.8203	0.6447	0.7220
	TabPFN	Low	0.8659	0.8401	0.8528
		Moderate	0.6727	0.7416	0.7054
		High	0.8175	0.7257	0.7689
	TabFM	Low	0.8687	0.8370	0.8526
		Moderate	0.6732	0.7488	0.7090
		High	0.8222	0.7280	0.7723

Table C.6: Mean absolute SHAP value of every feature with the full training data (average over the three classes)

Feature	CatBoost	TabPFN	TabFM
iron	0.7026	0.1271	0.1415
stress_level	0.6914	0.0844	0.0933
total_protein	0.6376	0.0729	0.0664
alt_liver	0.5675	0.0745	0.0779
vitamin_d	0.5610	0.0509	0.0652
calcium	0.4791	0.0511	0.0539
manganese	0.4072	0.0475	0.0520

Feature	CatBoost	TabPFN	TabFM
body_water_content	0.3379	0.0497	0.0425
family_hair_fall_history	0.2898	0.0217	0.0200
stress	0.2731	0.0342	0.0477
chemical_use	0.2698	0.0355	0.0276
anemia	0.2492	0.0422	0.0508
age	0.2483	0.0337	0.0296
chronic_illness	0.2377	0.0325	0.0297
late_night_sleep	0.1864	0.0381	0.0435
sleep_disturbance	0.1722	0.0280	0.0314
water_reason	0.1519	0.0038	0.0036
gender	0.1226	0.0158	0.0192
total_keratine	0.0308	0.0013	0.0035
hair_texture	0.0236	0.0022	0.0028

The values are not comparable between the columns (Section 4.3.2). Features are listed in the order of the CatBoost ranking.

Table C.7: Settings and run time of the nine SHAP analyses

Model	Training rows	Explainer	Records explained	Samples per record	Background centres	Time (s)
CatBoost	500	TreeExplainer	1000	-	-	0.3
CatBoost	2,000	TreeExplainer	1000	-	-	0.3
CatBoost	17,284	TreeExplainer	1000	-	-	0.3
TabPFN	500	KernelExplainer	30	100	10	65.8
TabPFN	2,000	KernelExplainer	30	100	10	193.0
TabPFN	17,284	KernelExplainer	30	100	10	3,251.3
TabFM	500	KernelExplainer	20	100	5	158.0
TabFM	2,000	KernelExplainer	20	100	5	581.8
TabFM	17,284	KernelExplainer	20	100	5	8,491.4

APPENDIX D

DATASET DESCRIPTION AND SAMPLE RECORDS

The dataset (data.csv) has 21,606 records and 23 columns. The columns id and full_name were removed before modelling, and hair_fall is the target. The 20 predictors are listed in Table 3.2. Table D.1 shows the first three records of the dataset without the name column, with one column per record.

Table D.1: First three records of the dataset (full_name omitted; one column per record)

Column	Record 1	Record 2	Record 3
id	1	2	3
age	39	26	34
gender	Female	Male	Male
total_protein	7.5	7.1	7.6
calcium	9.8	9.5	9.4
iron	56	124	114
vitamin_d	26.8	29.4	23.4
alt_liver	18	29	36
manganese	9.69	10.36	7.74
body_water_content	55.8	46.5	55.4
stress_level	23	31	19
total_keratine	50	18	42
hair_texture	48	76	62
family_hair_fall_history	0	1	1
chronic_illness	0	1	1
late_night_sleep	0	0	0
sleep_disturbance	0	1	1
water_reason	1	0	0
chemical_use	0	0	1
anemia	0	0	0
stress	0	1	0
hair_fall	0	2	1